

1.MySql基础

1.什么是关系型数据库？

关系型数据库（RDB，Relational Database）就是一种建立在关系模型基础上的数据库。关系模型表明了数据库中所存储的数据之间的联系（一对一、一对多、多对多）。

关系型数据库中，我们的数据都被存放在各种表中（比如用户表），表中的每一行就存放着一条数据（比如一个用户的信息）。

大部分关系型数据库都支持事务的四大特性(ACID)。

2.关系型和非关系型数据库的区别？

关系型数据库的优点

- 容易理解，因为它采用了关系模型来组织数据。
- 完全支持事务的ACID
- 可以保持数据的一致性。
- 数据更新的开销比较小。
- 支持复杂查询（带 where 子句的查询）

非关系型数据库（NOSQL）的优点

- 无需经过 SQL 层的解析，读写效率高。
- 基于键值对，读写性能很高，易于扩展
- 可以支持多种类型数据的存储，如图片，文档等等。
- 扩展（可分为内存性数据库以及文档型数据库，比如 Redis，MongoDB，HBase 等，适合场景：数据量大高可用的日志系统/地理位置存储系统）。

2.什么是MySQL

MySQL是一个关系型数据库，它采用表的形式来存储数据,有表结构（行和列）。行代表每一行数据，列代表该行中的每个值。列上的值是有数据类型的，比如：整数、字符串、日期等等。

3.数据库的三大范式

第一范式1NF(保证原子性)

确保数据库表字段的原子性。如果一个表中的“地址”列包含了省、市和具体地址，这不符合1NF，因为“地址”可以继续拆分成更细致的部分。按1NF要求，应该将“地址”拆分为“省份”、“城市”和“具体地址”等列。

第二范式2NF(消除部分依赖)

首先要满足第一范式，另外包含两部分内容，一是表必须有一个主键；二是非主键列必须完全依赖于主键，而不能只依赖于主键的一部分。例如，在学生成绩表中，如果学号和课程编号组成联合主键，那么成绩这一非主键列应同时依赖于学号和课程编号，而不是仅依赖于其中之一。

第三范式3NF（消除传递依赖）

首先要满足第二范式，另外非主键列必须直接依赖于主键，不能存在传递依赖。即不能存在：非主键列 A 依赖于非主键列 B，非主键列 B 依赖于主键的情况。例如，如果一个表中包含学生信息和院校信息，其中学生所在院校依赖于学号，而院校地址和电话又依赖于所在院校，这就形成了传递依赖。按照3NF的要求，需要将表拆分为学生表和院校表，使每个属性都直接依赖于主键。

2NF和3NF的区别？

- 2NF解决主键部分依赖问题
- 3NF解决主键传递依赖问题

4.SQL语句完整的执行顺序

FROM 子句组装来自不同数据源的数据；

WHERE 子句基于指定的条件对记录行进行筛选；

GROUP BY 子句将数据划分为多个分组；

使用聚集函数进行计算；

使用 HAVING 子句筛选分组；

计算所有的表达式；

SELECT 的字段；

使用 ORDER BY 对结果集进行排序。

5.having和where区别？

- 二者作用的对象不同，where 子句作用于表和视图，having 作用于组。
- where 在数据分组前进行过滤，having 在数据分组后进行过滤。
- 另一方面，HAVING 子句中不能使用除了分组字段和聚合函数之外的其他字段
- 先执行where,然后group by 然后having

5.Mysql的架构/组成

MySQL主要分为 Server 层和存储引擎层：

- **Server 层**：主要包括连接器、查询缓存、分析器、优化器、执行器等，所有跨存储引擎的功能都在这一层实现，比如存储过程、触发器、视图，函数等，还有一个通用的日志模块 binlog 日志模块。
- **存储引擎**：实现数据的存储和提取，不同的存储引擎如InnoDB、MyISAM通过这些API与Server层交互。存储引擎层提供数据存储和提取的具体实现，而Server层通过API调用这些功能，使得不同存储引擎能够与Server层无缝对接。

Server 层基本组件

- **连接器**：负责与客户端建立连接，并管理连接的权限验证和维持。当客户端连接 MySQL 时，server层会对其进行身份认证和权限校验。
- **查询缓存**：执行查询语句的时候，会先查询缓存，先校验这个 select语句 是否执行过，如果有缓存这个 sql，就会直接返回给客户端，如果没有命中，就会执行后续的操作。
- **分析器**：没有命中缓存的话，SQL 语句就会经过分析器，主要分为两步，词法分析和语法分析。首先通过词法分析识别出各个字符串的意义，然后进行语法分析，检查语句是否符合语法结构。如有错误，会提示错误位置。
- **优化器**：优化器对查询进行优化，选择最有效的查询路径，包括索引的选择和表的读取顺序，生成执行计划，包括选择合适的索引、表扫描顺序等。
- **执行器**：根据优化后的执行计划调用存储引擎的API来实际执行SQL语句。在执行前会再次检查权限，确认当前用户有权执行该操作，然后通过存储引擎接口访问数据。

6.sql语句在Mysql中的执行过程

其实我们的 SQL 可以分为两种，一种是查询，一种是更新（增加，修改，删除）

1.查询语句执行流程

大概有4步：

- 先连接器检查该语句是否有权限，如果没有权限，直接返回错误信息，如果有权限，会先查询缓存，以这条 SQL 语句为 key 在内存中查询是否有结果，如果有直接返回缓存结果，如果没有，执行下一步。
- 通过分析器进行词法分析，提取 SQL 语句的关键元素。然后判断这个 SQL 语句是否有语法错误，比如关键词是否正确等等，如果检查没问题就执行下一步。
- 接下来就是优化器进行确定执行方案，优化器根据自己的优化算法进行选择执行效率最好的一个方案
- 进行权限校验，如果没有权限就会返回错误信息，如果有权限就会调用数据库引擎接口，返回引擎的执行结果。

2.更新语句执行流程

举个例子，更新语句是这样的：

```
update user set name = '张三' where id = 1;
```

- 1.先查询到 id 为1的记录，有缓存会使用缓存。
- 2.拿到查询结果，将 name 更新为张三，然后调用引擎接口，写入更新数据，innodb 引擎将数据保存在内存中，同时记录 redo log，此时 redo log 进入 准备 状态。
- 3.执行器收到通知后记录 binlog，然后调用引擎接口，提交 redo log 为提交状态。
- 4.更新完成。

为什么记录完 redo log，不直接提交，而是先进入 准备 状态？

假设先写 redo log 直接提交，然后写 binlog，写完 redo log 后，机器挂了，binlog 日志没有被写入，那么机器重启后，这台机器会通过 redo log 恢复数据，但是这个时候 binlog 并没有记录该数据，后续进行机器备份的时候，就会丢失这一条数据，同时主从同步也会丢失这一条数据。

7.inner join,left join和right join的区别

类型 作用

- 1 Join(Inner Join)内连接 查出两表完全匹配的部分。（交集）
- 2 Left Join左连接 返回左表所有的行，右表返回匹配行，不匹配的返回NULL
- 3 Right Join右连接 返回由表所有的行，左表返回匹配行，不匹配的返回NULL
- 4 Full Join全连接 只要其中一个表存在匹配，则返回行

8.MySQL 是如何保证数据不丢失的？

- 只要redolog 和 binlog 保证持久化磁盘就能确保MySQL异常重启后回复数据
- 在恢复数据时，redolog 状态为 commit 则说明 binlog 也成功，直接恢复数据；如果 redolog 是 prepare，则需要查询对应的 binlog事务是否成功，决定是回滚还是执行。

9.为什么 MySQL 自增主键 ID 不连续？

MySQL 自增主键 ID 不连续的原因可能有以下几点：

1. 插入数据失败：当插入数据时，如果某些原因导致插入失败，那么自增 ID 会跳过这个值，导致不连续。
2. 删除数据：当删除某条记录时，自增 ID 不会回退，因此会导致不连续。
3. 事务回滚：在事务中插入数据时，如果发生错误导致事务回滚，那么已经分配的自增 ID 也会被跳过，导致不连续。
4. 数据库备份和恢复：在进行数据库备份和恢复时，可能会涉及到数据的重新分配，导致自增 ID 不连续。
5. 分布式系统：在分布式系统中，多个节点可能同时分配自增 ID，虽然每个节点内部是连续的，但在整个系统中可能会出现不连续的情况。
6. 手动设置自增 ID：如果在插入数据时手动设置了自增 ID，可能会导致不连续。

为了避免自增 ID 不连续的问题，可以采取以下措施：

1. 使用 UUID 作为主键，而不是自增 ID。
2. 在不影响性能的前提下，尽量减少事务回滚和数据删除操作。
3. 在分布式系统中，使用全局唯一 ID 生成策略，如雪花算法（Snowflake）等。

10.orderby 排序内部原理

在 MySQL 中，`ORDER BY` 用于对查询结果进行排序。它有两种排序方式：

1. 全字段排序：按照查询语句中 `ORDER BY` 后面的字段进行排序。这种方式比较直观，直接按照指定的字段排序即可。
2. RowNumber 排序：当 `ORDER BY` 后面没有指定具体的字段时，MySQL 会采用 RowNumber 排序方式。这种方式是 MySQL 内部实现的排序机制，具体实现过程如下：
 - 首先，MySQL 会根据表的大小，分配一块内存空间（称为 Sort Buffer），用于存放需要排序的数据。
 - 然后，MySQL 会将表中的数据按照 RowNumber 的顺序，依次读取到 Sort Buffer 中。这个过程是通过多路归并排序算法实现的，即将表中的数据分成多个小的子集，每个子集内部按照 RowNumber 排序，然后再将子集合并成一个有序的大集合。
 - 最后，MySQL 会按照 Sort Buffer 中的数据顺序，依次输出结果。

需要注意的是，由于 Sort Buffer 的大小是有限的，因此在处理大表时，RowNumber 排序可能会导致内存不足的问题。此时，可以考虑使用外部排序的方式，即通过磁盘文件进行排序操作。

11.什么是sql注入？怎么解决？

SQL注入的原理是将SQL代码伪装到输入参数中，传递到服务器解析并执行的一种攻击手法。也就是说，在一些对SERVER端发起的请求参数中植入一些SQL代码，SERVER端在执行SQL操作时，会拼接对应参数，同时也将一些SQL注入攻击的“SQL”拼接起来，导致会执行一些预期之外的操作

如何解决SQL注入

- 严格的参数校验
参数校验就没得说了，在一些不该有特殊字符的参数中提前进行特殊字符校验即可。
- **PreparedStatement预编译防止SQL注入**
PreparedStatement具有预编译功能，以上述SQL为例
使用PreparedStatement预编译后的SQL为：

```
delete from table1 where id= ?
```

此时SQL语句结构已固定，无论"?"被替换为任何参数，SQL语句只认为where后面只有一个条件，当再传入 1001 or 1 = 1时当作一个整体，语句会报错，从而达到防止SQL注入效果

• mybatis中#{ }防止SQL注入

mybatis中#{ }表达式防止SQL注入与PreparedStatement类似，都是对SQL语句进行预编译处理

注意：

#{ }：参数占位符，可防sql注入

\${ }：拼接替换符，不能防止SQL注入，一般用于

- 传入数据库对象（如：数据库名称、表名）
- order by 后的条件

2.MySQL优化

-1.对数据库优化的理解

MySQL数据库优化是多方面的，原则是减少系统的瓶颈，减少资源的占用，增加系统的反应速度。例如，通过优化文件系统，提高磁盘I/O的读写速度；通过优化操作系统调度策略，提高MySQL在高负荷情况下的负载能力；优化表结构、索引、查询语句等使查询响应更快。

- 针对查询，我们可以通过使用索引、使用连接代替子查询的方式来提高查询速度。
- 针对慢查询，我们可以通过分析慢查询日志，来发现引起慢查询的原因，从而有针对性的进行优化。
- 针对插入，我们可以通过禁用索引、禁用检查等方式来提高插入速度，在插入之后再启用索引和检查。
- 针对数据库结构，我们可以通过将字段很多的表拆分成多张表、增加中间表、增加冗余字段等方式进行优化。

-1.数据表结构设计优化

- 不能完全按照数据库三大范式设计。数据库范式就是让一个字段干一个字段的事，一个表干一个表的事，拒绝冗余。但如果完全没有冗余的话，很多时候会引起多表join降低性能，适当增加冗余可以提高查询性能。比如要查询班级里学生的姓名，如果在班级表里冗余学生字段的话就不用去join查询学生表了，一般我们冗余的事不太容易发生变化的字段比如姓名性别等；
- 当不可避免需要join的时候，可以设计个中间表去存储所有数据，用了中间表就要考虑到这张表的更新机制避免数据不一致的情况（最好定期轮询检查）
- 要设计一个游标字段（自增主键和时间戳很适合当游标字段），减少limit查询(limit n,m)尽量依靠游标来查，比如说你用主键，下次查询就>或<上次查询的最后一个id就可以了

0.如何分析sql的性能(EXPLAIN 命令)

我们可以使用 EXPLAIN 命令来分析 SQL 的 执行计划 。

1.什么是执行计划

执行计划 是指一条 SQL 语句在经过 **MySQL 查询优化器** 的优化会后，具体的执行方式。

通过 EXPLAIN 的结果，可以了解到如数据表的查询顺序、数据查询操作的操作类型、哪些索引可以被命中、哪些索引实际会命中、每个数据表有多少行记录被查询等信息。

1 explain select * from employee where name = '捡田螺的小男孩';

信息

结果 1

剖析

状态

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employee	(Null)	ref	idx_name_age	idx_name_age	768	const	1	100 (Null)	

@稀土掘金技术社区

2. 执行计划常用字段

MySQL 为我们提供了 `EXPLAIN` 命令，来获取执行计划的相关信息。

需要注意的是，`EXPLAIN` 语句并不会真的去执行相关的语句，而是通过查询优化器对语句进行分析，找出最优的查询方案，并显示对应的信息。

执行计划有12个字段组成，常用的有：

1.type表示查询表连接类型**，查看索引执行情况的一个重要指标。以下性能从好到坏依次：`system`

`> const > eq_ref > ref > ref_or_null > index_merge > unique_subquery >`

`index_subquery > range > index > ALL`

- `system`：这种类型要求数据库表中只有一条数据，是 `const` 类型的一个特例，一般情况下是不会出现的。
- **`const`：通过一次索引就能找到数据，一般用于主键或唯一索引作为条件，这类扫描效率极高，速度非常快。**基于主键或唯一索引唯一值查询，最多返回一条结果，比`eq_ref`略好。
- `eq_ref`：常用于主键或唯一索引扫描，一般指使用主键的关联查询
- `ref`：常用于非主键和唯一索引扫描。
- `ref_or_null`：这种连接类型类似于 `ref`，区别在于 MySQL 会额外搜索包含 `NULL` 值的行
- **`index_merge`：使用了索引合并优化方法，查询使用了两个以上的索引。**
- `unique_subquery`：类似于 `eq_ref`，条件用了 `in` 子查询
- `index_subquery`：区别于 `unique_subquery`，用于非唯一索引，可以返回重复值。
- `range`：常用于范围查询，比如：`between ... and` 或 `In` 等操作
- **`index`：全索引扫描（通过遍历整个索引树来获取数据行，而不是直接扫描整张表的数据。）**
- **`ALL`：全表扫描**

2.possible_keys:表示 MySQL 执行查询时可能用到的索引。如果这一列为 `NULL`，则表示没有可能用到的索引；这种情况下，需要检查 `WHERE` 语句中所使用的列，看是否可以通过给这些列中某个或多个添加索引的方法来提高查询性能。

3.key:表示 MySQL 实际使用到的索引。如果为 `NULL`，则表示未用到索引。

4.key_len:表示 MySQL 实际使用的索引的最大长度；当使用到联合索引时，有可能是多个列的长度和。在满足需求的前提下越短越好。如果 `key` 列显示 `NULL`，则 `key_len` 列也显示 `NULL`。

5.rows:估算要找到所需的记录，需要读取的行数，这是一个估计值。

6.extra:这列包含了 MySQL 解析查询的额外信息，通过这些信息，可以更准确的理解 MySQL 到底是如何执行查询的。常见的值如下：

- `Using filesort`：在排序时使用了外部的索引排序，没有用到表内索引进行排序。
- **`Using temporary`**：MySQL 需要创建临时表来存储查询的结果，常见于 `ORDER BY` 和 `GROUP BY`。
- **`Using index`：表明查询使用了覆盖索引，不用回表，查询效率非常高。**
- **`Using index condition`**：表示查询优化器选择使用了索引条件下推这个特性。
- **`Using where`**：表明查询使用了 `WHERE` 子句进行条件过滤。一般在没有使用到索引的时候会出现。
- **`Using join buffer (Block Nested Loop)`**：连表查询的方式，表示当被驱动表的没有使用索引的时候，MySQL 会先将驱动表读出来放到 `join buffer` 中，再遍历被驱动表与驱动表进行查询。

3.explain具体怎么分析一条慢sql?

如果一条sql执行很慢的话，我们通常会使用mysql自动的执行计划explain来查看这条sql的执行情况，

比如在这里面可以通过key和key_len检查是否命中了索引，如果本身已经添加了索引，也可以判断索引是否有失效的情况，

第二个，可以通过type字段查看sql是否有进一步的优化空间，是否存在全索引扫描或全盘扫描，

第三个可以通过extra建议来判断，是否出现了回表的情况，如果出现了，可以尝试添加索引或修改返回字段来修复

1.慢查询问题

0.接口响应慢怎么办？

您可以尝试以下方法来优化Java接口的性能：

1. **代码优化**：审查接口代码，确保它们没有不必要的循环、嵌套或递归，尽可能减少资源消耗和时间复杂度。
2. **数据库优化**：如果接口涉及数据库操作，可以优化SQL查询，添加索引以提高检索速度，或者考虑使用缓存来减少数据库访问次数。
3. **并发处理**：使用多线程或异步处理来处理并发请求，提高系统吞吐量和响应速度。
4. **资源管理**：及时释放资源，如关闭数据库连接、IO流等，以避免资源泄露导致性能下降。
5. **缓存**：考虑将频繁使用的数据缓存起来，减少重复计算或查询的开销。
6. **网络优化**：确保网络连接稳定，并尽量减少网络通信的延迟。
7. **代码审查和性能测试**：定期进行代码审查和性能测试，及时发现并解决潜在的性能问题。

以上是一些常见的优化方法，根据具体情况，您可能需要结合实际场景进行调整和优化。

1.怎么定位慢查询

2种方法：

- **运维监控工具Slywalking**

我们当时做压测的时候有的接口非常的慢，接口的响应时间超过了2秒以上，因为我们当时的系统部署了运维的监控系统Skywalking，在展示的报表中可以看到是哪一个接口比较慢，并且可以分析这个接口哪部分比较慢，这里可以看到SQL的具体的执行时间，所以可以定位是哪个sql出了问题

- **mysql慢查询日志**
 - **开启慢查询日志功能**：慢查询日志是MySQL内置的一项功能，可以记录执行超过指定时间的SQL语句。在MySQL配置文件中设置 `slow_query_log=1` 以启用该功能，并设定 `long_query_time=2` 来规定只有执行时间超过2秒的SQL才会被记录。
 - **分析慢查询日志文件**：通过查看日志文件中记录的SQL语句和相应的执行时间、扫描行数等指标，可以直观地识别出慢查询。一般情况下，慢查询日志会记录查询执行的时间、返回的行数以及检查的行数等重要信息。

2.导致MySQL慢查询有哪些原因？

1. 索引使用不当或缺失：

- 索引未覆盖查询中使用的字段，或者索引被查询中的函数调用、范围查询或非等值条件所避开。
- 索引选择性差，即索引不能有效地过滤掉大量行。

2. 单表数据量太大

3. 查询使用了临时表

4. 表结构设计不佳：

- 过度规范化或反规范化可能导致额外的JOIN操作，增加查询复杂度。
- 冗余数据或设计不良的外键关系。

5. limit深度分页问题

6. 复杂的查询语句：

- 包含大量的连接（JOINS）、子查询或者复杂的嵌套表达式，这些都会增加CPU和I/O负载。

3.大表查询慢常见优化措施？

当MySQL单表记录数过大时，数据库的性能会明显下降，一些常见的优化措施如下：

- **创建适当的索引：**可根据EXPLAIN来查看是否用了索引还是全表扫描
 - **索引类型选择：**根据查询的需要创建有针对性的索引，比如在WHERE和ORDER BY命令上涉及的列建立索引。
 - **避免索引过多：**索引并不是越多越好，需要根据实际查询有针对性地创建，同时删除不必要的索引，避免维护索引的额外开销。
 - **合理设置索引顺序：**使用多列索引时，注意索引的顺序与查询条件保持一致，可以提高索引的使用效率。
 - 避免索引失效
- **优化查询语句：**
 - **避免全表扫描：**应尽量减少在查询语句中使用全表扫描的操作，比如避免使用 SELECT，尽量在 WHERE 子句中使用已有索引的字段。
 - **优化子查询和连接：**尽量减少子查询的使用，将其转化为连接（JOIN）方式；如果必须使用子查询，尽量使其返回更少的记录数。同样地，**避免多个表的连接，特别是大数据表之间的连接。**
 - **使用具体的列名：**在SELECT语句中，指定所需的具体列名而不是使用 *，可以避免回表查询
- **利用缓存：**利用Redis等缓存热点数据，提高查询效率
- **提升硬件配置**
 - **性能更高的硬件设备：**采用更快的存储介质（如固态硬盘）和更大的内存容量，以提升查询的IO性能。
 - **均衡系统资源：**合理分配系统资源，确保MySQL的服务模型能够充分利用服务器的CPU资源。
- 读写分离。经典的数据库拆分方案，主库负责写，从库负责读
- 通过分库分表的方式进行优化，主要有垂直拆分和水平拆分

4.慢sql的优化分析思路？

步骤如下：

1.查看慢查询日志记录，分析慢SQL

通过慢查询日志slow log，定位那些执行效率较低的SQL语句，重点关注分析

2.explain查看分析SQL的执行计划

当定位出查询效率低的SQL后，可以使用 explain 查看 SQL 的执行计划。

比如在这里面可以通过key和key_len检查是否命中了索引，如果本身已经添加了索引，也可以判断索引是否有失效的情况，

第二个，可以通过type字段查看sql是否有进一步的优化空间，是否存在全索引扫描或全盘扫描，

第三个可以通过extra建议来判断，是否出现了回表的情况，如果出现了，可以尝试添加索引或修改返回字段来修复

3.profile 分析执行耗时

explain 只是看到 SQL 的预估执行计划，如果要了解 SQL 真正的执行线程状态及消耗的时间，需要使用 profiling。开启 profiling 参数后，后续执行的 SQL 语句都会记录其资源开销，包括 IO，上下文切换，CPU，内存 等等，我们可以根据这些开销进一步分析当前慢SQL的瓶颈再进一步进行优化。

4.Optimizer Trace分析详情

profile只能查看到SQL的执行耗时，但是无法看到SQL真正执行的过程信息，即不知道MySQL优化器是如何选择执行计划。这时候，我们可以使用 Optimizer Trace，它可以跟踪执行语句的解析优化执行的全过程。

大家可以查看分析其执行树，会包括三个阶段：

- join_preparation：准备阶段
- join_optimization：分析阶段
- join_execution：执行阶段

5.确定问题并采用相应的措施

最后确认问题，就采取对应的措施。

- 多数慢SQL都跟索引有关，比如不加索引，索引不生效、不合理等，这时候，我们可以**优化索引**。
- 我们还可以优化SQL语句，比如一些in元素过多问题（分批），深分页问题（基于上一次数据过滤等），进行时间分段查询
- SQL没办法很好优化，可以改用ES的方式，或者数仓。
- 如果单表数据量过大导致慢查询，则可以考虑分库分表
- 如果数据库在刷脏页导致慢查询，考虑是否可以优化一些参数，跟DBA讨论优化方案
- 如果存量数据量太大，考虑是否可以让部分数据归档

2.sql优化经验

sql语句优化小技巧

SQL语句优化

- **查询时只返回必要的列，用具体的字段列表代替 select * 语句,因为要尽量用聚集索引防止回表查询**
- **SQL语句要避免造成索引失效的写法**
- 尽量用union all代替union UNION 会进行去重处理，这会增加排序和比较的计算成本
- 避免在where子句中对字段进行表达式操作
- Join优化 能用inner join 就不用left join right join，如必须使用 一定要以小表为驱动，（内连接会对两个表进行优化，优先把小表放到外边，把大表放到里边。left join 或 right join，不会重新调整顺序）
 - inner join 内连接，只保留两张表中完全匹配的结果集；

- left join会返回左表所有的行，即使在右表中没有匹配的记录；
- right join会返回右表所有的行，即使在左表中没有匹配的记录；
- 使用varchar代替char.(因为可变字符串存储空间小，可节省空间)
- 将多次插入换成批量Insert插入：例如，使用 `INSERT INTO order (id, code, user_id) VALUES (123, '001', 100), (124, '002', 100), (125, '003', 101);`
- 避免在where子句中使用!=或<>操作符
 - 使用 != 和 <> 很可能会让索引失效
 - 应尽量避免在 where 子句中使用 != 或 <> 操作符，否则引擎将放弃使用索引而进行全表扫描
- 多次插入改成批量插入
 - 默认新增SQL有事务控制，导致每条都需要事务开启和事务提交，而批量处理是一次事务开启和提交，效率提升明显，达到一定量级，效果显著，平时看不出来。

sql优化的真实场景

1.正确建立索引（联合索引）

应对场景：在实际工作中，对于一个承载着业务核心的表，它除了数据条数多之外，而且会有很多列。对于这样核心的表，我们又频繁的有查询修改操作。查询的时候 往往是多个列在一起构成筛选条件,所以针对这种情况我们就需要创建合理的 联合索引。

这里通过创建一个用户信息表来讲解，表名为userbase 表信息如下：

userID	age	score	name	sex
1	10	98	小明	男
2	11	94	毛毛	男
3	10	100	红红	女
4	15	50	丹丹	女

创建一个联合索引，由 age,score,sex 组成

注意他们的前后顺序：age 是1，score是2，sex是3

执行不同条件查询及结果：

语句：select * from userbase where age =10 and score=98 and sex='男' (顺序：1、2、3)

结果：使用了索引

语句：select * from userbase where age =10 and score=98 (顺序：1、2)

结果：使用了索引语句：

select * from userbase where score=98 and sex='男' (顺序：2、3)

结果：未使用索引

语句：select * from userbase where score=98 (顺序：2)

结果：未使用索引

联合索引的最左原则：以最左边的那列为中心，只要它参与了，就可以让索引起到作用，否则就不起作用。

我们知道了最左原则，就可以避免创建过多而无用的索引。索引的创建，会占用硬盘大量的空间，所以合理的创建索引，不但可以让我们查询效率提高，也能减少硬盘空间。

2.建表时选择合适的数据类型

使用varchar代替char.(因为可变常字段存储空间小, 可节省空间)

3.不滥用事务修改事务级别

不同的数据库, 有自己默认的事务类型。

我们在做开发时, 因为某些关键业务比如涉及到金钱的转账等, 必须用事务防止出现脏数据(或脏读现象), 保证数据的准确性。但是这样的情况下, 必定有损执行性能, 在一些不必要的业务中, 建议取消事务。

如我要查询某个结果, 这个结果即使出现脏读, 也不影响我们业务。就可以取消事务, 来提高整体效率。

4.避免不必要的数据库连接

如果要执行100行[sql语句](#), 如果可以一次执行, 那就在一次数据库连接, 一起执行100条。而不是通过100次数据库连接, 每次连接执行一条。较少数据库连接时产生的性能消耗。

在Java中, 我们可以使用JDBC (Java Database Connectivity) 来连接和操作数据库。为了一次性执行多条SQL语句, 我们可以使用Statement或PreparedStatement对象的executeBatch()方

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "username";
        String password = "password";
        try {
            Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement();
            for (int i = 1; i <= 100; i++) {
                stmt.addBatch("INSERT INTO mytable VALUES (" + i + ", 'value" + i
+ "')");
            }
            int[] results = stmt.executeBatch();
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

在这个例子中, 我们首先建立了一个到数据库的连接, 然后创建了一个Statement对象。然后, 我们在循环中添加了100个INSERT语句到批处理中。最后, 我们调用executeBatch()方法一次执行所有的插入操作, 然后关闭连接。

3.分页问题

1.MySQL查询 limit 1000,10 和limit 10 速度一样快吗？

结论：mysql查询中 limit 1000,10 会比 limit 10 更慢。原因是 limit 1000,10 会取出1000+10条数据，并抛弃前1000条，这部分耗时更大。

分析背后原理：

limit 1000,10 和limit 10,对应 `limit offset, size` 和 `limit size` 两种方式。

而其实 `limit size`，相当于 `limit 0, size`。也就是从0开始取size条数据。

也就是说，两种方式的**区别在于offset是否为0**。

先来看下limit sql的内部执行逻辑。

MySQL内部分为**server层**和**存储引擎层**。一般情况下存储引擎都用innodb。

执行器可以通过调用存储引擎提供的接口，将一行行数据取出，当这些数据完全符合要求（比如满足其他where条件），则会放到**结果集中**，最后返回给调用mysql的**客户端**。

以主键索引的limit执行过程为例：

执行 `select * from xxx order by id limit 0, 10;`，select后面带的是**星号**，也就是要求获得行数据的**所有字段信息**。

server层会调用innodb的接口，在innodb里的主键索引中获取到第0到10条**完整行数据**，依次返回给server层，并放到server层的结果集中，返回给客户端。

把offset搞大点，比如执行的是：`select * from xxx order by id limit 500000, 10;`

server层会调用引擎层innodb的接口，由于这次的offset=500000，会在innodb里的主键索引中获取到第0到（500000 + 10）条完整行数据，返回给server层之后根据offset的值挨个抛弃，最后只留下最后面的size条，也就是10条数据，放到server层的结果集中，返回给客户端。

可以看出，**当offset非0时，server层会从引擎层获取到很多无用的数据，而获取的这些无用数据都是要耗时的。**

因此，mysql查询中 limit 1000,10 会比 limit 10 更慢。原因是 limit 1000,10 会取出1000+10条数据，并抛弃前1000条，这部分耗时更大。

2.深度分页怎么优化？

(什么是深度分页问题？)

`select * from xxx order by id limit 500000, 10;`

当offset非常大时，server层会从引擎层获取到很多无用的数据，而当select后面是*号时，就需要拷贝完整的行信息，**拷贝完整数据**相比**只拷贝行数据里的其中一两个列字段**更耗费时间。这就是深度分页问题。

优化方法：

- **子查询优化**

我们可以采用**覆盖索引和子查询**来解决

先分页查询数据的id字段，确定了id之后，再用子查询来过滤，只查询这个id列表中的数据就可以了

因为查询id的时候，走的覆盖索引，所以效率可以提升很多

```
select * from xxx where id >=(select id from xxx order by id limit 500000, 1)
order by id limit 10;
```

- 延迟关联

尽可能地使用索引覆盖扫描，而不是查询所有的列，然后根据需要做一次关联操作再返回所需的列。

```
select * from xxx table1 inner join (select id from xxx order by id limit
500000,10) as table2 on table1.id = table2.id;
```

4.列举一下，常用的数据库设计优化技巧？

- 字段尽量避免使用NULL
- 合理选择数据类型
- 字段选择合适的长度
- 正确使用索引
- 尽量少定义text类型
- 合理的数据表结构设计
- 适当的冗余设计
- 优化SQL查询语句
- 一张表的字段不宜过多

5.group by查询效率慢,如何优化

group by详解链接：[看一遍就理解：group by详解\(qq.com\)](#)

group by一般用于分组统计，它表达的逻辑就是根据一定的规则，进行分组。日常开发中，我们使用得比较频繁。如果不注意，很容易产生慢SQL。

group by可能会慢在哪里？因为它既用到临时表，又默认用到排序。有时候还可能用到磁盘临时表。

- 如果执行过程中，会发现内存临时表大小到达了上限（控制这个上限的参数就是tmp_table_size），会把内存临时表转成磁盘临时表。
- 如果数据量很大，很可能这个查询需要的磁盘临时表，就会占用大量的磁盘空间。

如何优化group by呢？

- group by后面的字段加索引
- order by null不用排序
- 尽量只使用内存临时表
- 使用SQL_BIG_RESULT

6.order by查询效率慢,如何优化.

order by详解链接：[看一遍就理解：order by详解\(qq.com\)](#)

在MySQL中，ORDER BY用于对查询结果进行排序。它有两种排序方式：

1. 全字段排序：按照查询语句中ORDER BY后面的字段进行排序。这种方式比较直观，直接按照指定的字段排序即可。

2. RowNumber 排序：当 `ORDER BY` 后面没有指定具体的字段时，MySQL 会采用 RowNumber 排序方式。这种方式是 MySQL 内部实现的排序机制，具体实现过程如下：

- 首先，MySQL 会根据表的大小，分配一块内存空间（称为 Sort Buffer），用于存放需要排序的数据。
- 然后，MySQL 会将表中的数据按照 RowNumber 的顺序，依次读取到 Sort Buffer 中。这个过程是通过多路归并排序算法实现的，即将表中的数据分成多个小的子集，每个子集内部按照 RowNumber 排序，然后再将子集合并成一个有序的大集合。
- 最后，MySQL 会按照 Sort Buffer 中的数据顺序，依次输出结果。

`order by` 排序，分为全字段排序和 `rowid` 排序。它是拿 `max_length_for_sort_data` 和结果行数据长度对比，如果结果行数据长度超过 `max_length_for_sort_data` 这个值，就会走 `rowid` 排序，相反，则走全字段排序。

`rowid` 排序，一般需要回表去找满足条件的数据，所以效率会慢一点。如果是 `order by` 排序，可能会借助磁盘文件排序的话，效率就更慢一点。

如何优化 `order by` 的文件排序？

- 因为数据是无序的，所以需要排序。如果数据本身是有序的，那就不会再用文件排序啦。而索引数据本身是有序的，我们通过建立索引来优化 `order by` 语句。
- 我们还可以通过调整 `max_length_for_sort_data`、`sort_buffer_size` 等参数优化；

7.原本可以执行得很快的 SQL 语句，执行速度却比预期的慢很多，原因是什么？如何解决？

原因：从大到小可分为四种情况

- MySQL 数据库本身被堵住了，比如：系统或网络资源不够。
- SQL 语句被堵住了，比如：表锁，行锁等，导致存储引擎不执行对应的 SQL 语句。
- 确实是索引使用不当，没有走索引。
- 表中数据的特点导致的，走了索引，但回表次数庞大。

解决：

- 考虑采用 `force index` 强行选择一个索引
- 考虑修改语句，引导 MySQL 使用我们期望的索引。比如把“`order by b limit 1`”改成“`order by b,a limit 1`”，语义的逻辑是相同的。
- 第三种方法是，在有些场景下，可以新建一个更合适的索引，来提供给优化器做选择，或删除错误的索引。
- 如果确定是索引根本没必要，可以考虑删除索引。

2.索引

-1.对索引的理解？

索引是一个单独的、存储在磁盘上的数据库结构，包含着对数据表里所有记录的引用指针。使用索引可以快速找出在某个或多个列中有一特定值的行，所有 MySQL 列类型都可以被索引，对相关列使用索引是提高查询操作速度的最佳途径。

索引是在存储引擎中实现的，因此，每种存储引擎的索引都不一定完全相同，并且每种存储引擎也不一定支持所有索引类型。MySQL 中索引的存储类型有两种，即 BTREE 和 HASH，具体和表的存储引擎相关。MyISAM 和 InnoDB 存储引擎只支持 BTREE 索引；MEMORY/HEAP 存储引擎可以支持 HASH 和 BTREE 索引。

0.如何创建及保存MySQL的索引？

- 在创建表的时候创建索引

可以按照如下方式，在id字段上使用UNIQUE关键字创建唯一索引

```
CREATE TABLE t1 (  
  id INT NOT NULL,  
  name CHAR(30) NOT NULL,  
  UNIQUE INDEX UniqIdx(id)  
);
```

- 在已存在的表上创建索引

例如，可以按照如下方式，在bookId字段上建立名称为UniqidIdx的唯一索引：

```
CREATE UNIQUE INDEX UniqidIdx ON book (bookId);
```

-2.全表扫描比遍历索引更高效的场景

在某些情况下，全表扫描可能比遍历索引更高效。这是因为数据库优化器会基于成本估算和统计数据来决定是否使用索引。具体来说，如果查询需要返回表中大部分数据，或者查询条件不足以显著减少需要检索的数据量，全表扫描可能会更快。

以一个例子来说明：假设有一张包含10万行数据的表，每个数据块可以存放100行数据，那么整张表大约有1000个数据块。如果一个查询需要返回20%的数据，也就是2万行，那么即使使用了索引，数据库仍然需要读取大量的数据块。在这种情况下，如果索引的选择性不高，即索引列中的值重复率较高，那么索引的优势就会减弱。因为索引本身也占用空间，且在索引中查找到每一行后，还需要回到表中获取完整的行数据，这个过程称为“回表”。如果回表的次数非常多，那么这个额外的开销就可能导致索引扫描不如全表扫描高效。

此外，全表扫描在某些情况下可以避免。例如，如果查询可以通过仅访问索引而不需要访问表数据来完成，那么全索引扫描（也称为快速全索引扫描）可能会更高效。这通常发生在查询只需要返回索引列的情况下。

总的来说，数据库优化器会根据多种因素决定是否使用索引，包括表的大小、索引的选择性、查询的复杂性以及数据分布等。在实际应用中，理解这些原理有助于编写更高效的SQL查询，并在必要时对索引策略进行调整。

-1.Hash索引和B+树索引的区别

hash索引底层就是hash表，进行查找时，调用一次hash函数就可以获取到相应的键值，之后进行回表查询获得实际数据。B+树底层实现是b+树，

哈希索引**不支持排序**，因为哈希表是无序的。

哈希索引**不支持范围查找**。

哈希索引**不支持模糊查询**及多列索引的最左前缀匹配。

因为哈希表中会**存在哈希冲突**，所以哈希索引的性能是不稳定的，而B+树索引的性能是相对稳定的，每次查询都是从根节点到叶子节点。

1.什么是索引

索引是一种用于快速查询和检索数据的数据结构，其本质可以看成是一种排序好的数据结构。它可以比作一本字典的目录，可以帮你快速找到对应的记录。

2.索引的优缺点

索引的优点主要有以下几条：

1. 通过创建唯一索引，可以保证数据库表中每一行数据的唯一性。
2. 可以大大加快数据的查询速度，这也是创建索引的主要原因。
3. 在实现数据的参考完整性方面，可以加速表和表之间的连接。
4. 在使用分组和排序子句进行数据查询时，也可以显著减少查询中分组和排序的时间。

增加索引也有许多不利的方面，主要表现在如下几个方面：

1. 创建索引和维护索引要耗费时间，并且随着数据量的增加所耗费的时间也会增加。
2. 索引需要占磁盘空间，除了数据表占数据空间之外，每一个索引还要占一定的物理空间，如果有大量的索引，索引文件可能比数据文件更快达到最大文件尺寸。
3. 当对表中的数据进行增加、删除和修改的时候，索引也要动态地维护，这样就降低了数据的维护速度。

3.索引的作用/为什么用索引

数据是存储在磁盘上的，查询数据时，如果没有索引，会加载所有的数据到内存，依次进行检索，读取磁盘次数较多。有了索引，就不需要加载所有数据，因为B+树的高度一般在2-4层，最多只需要读取2-4次磁盘，查询速度大大提升。

4.索引数据结构

索引的数据结构主要有B+树和哈希表。MySQL的默认的存储引擎InnoDB采用的B+树的数据结构来存储索引。

-1.b+树的叶子节点和非叶子节点分别存储的什么？

聚簇索引：

- **叶子节点：**存储的是整行数据。
- **非叶子节点：**存储的是主键值，这些值用于指导查询过程中的搜索方向。

非聚簇索引：

- **叶子节点：**存储的是主键值，而不是整行数据。这是因为非聚簇索引的目的是加速查询，而不是直接提供数据。当通过非聚簇索引找到主键值后，MySQL会进行所谓的“回表”操作，即根据主键值去聚簇索引中查找相应的整行数据。
- **非叶子节点：**存储的是非主键索引列的值，这些值也用于指导查询过程中的搜索方向。

B+树是一种平衡多路查找树，广泛应用于数据库和文件系统的索引结构。具体到MySQL的B+树索引结构：

- **非叶子节点：**这些节点不存储实际的数据记录，只保存索引字段（也就是键值）以及指向子节点的指针。这样做可以使得每个非叶子节点能够索引大量的数据，而不需要占用过多的空间。由于非叶子节点仅包含键值和指针，这减少了树的高度，从而加快了搜索速度。
- **叶子节点：**所有实际的数据都存储在叶子节点中。每个叶子节点不仅包含了索引字段，还包含了完整的数据记录。此外，叶子节点之间通过指针相连，形成了一个链表，这个特性优化了范围查询的性能。

总的来说，B+树索引结构在MySQL中的应用主要是为了提高数据检索的效率，特别是对于大量数据的查询操作。通过减少I/O操作的次数和利用磁盘预读特性，B+树能够有效地提升数据库系统的性能。

0.什么是b+树?

B+树是一种常用的数据结构，通常用于数据库索引和文件系统中。它是一种多路搜索树，具有以下特点：

1. 内部节点存储的是索引字段（也就是键值）以及指向子节点的指针，叶子节点存储的是数据记录；
2. 所有叶子节点之间通过指针连接成一个有序链表，便于范围查询；
3. 内部节点通常不存储数据，只存储键值和指向子节点的指针；
4. B+树的高度通常比较低，查找效率高；
5. B+树的叶子节点都在同一层，便于范围查询和顺序遍历。

1.b+树和 b树的区别

B树（B-树）和B+树都是数据库和文件系统中常用的数据结构，它们在索引优化和存储管理方面具有重要作用。B+树相对于B树进行了优化，使其更加适合作为外存储的索引结构。具体分析如下：

1. 节点结构

- **B树**：每个节点既包含键也包含数据值。每个非叶子节点中的键和数据都紧密存储在一起，这可能导致在数据量较大时树的高度增加，从而影响查询效率。
- **B+树**：非叶节点只包含键，不包含实际数据，所有数据项都存储在叶子节点上。叶子节点之间通过指针相连，形成链表，便于区间查找和遍历。

2. 磁盘IO次数

- **B树**：因为数据和键都存储在每一个节点中，当数据量大时树的高度可能较高，导致更多的磁盘IO操作。
- **B+树**：由于非叶节点仅存储键，能够存放更多的键，从而降低树的高度，减少磁盘IO次数，提高查询性能。

3. 查询性能

- **B树**：查询性能不稳定，最好情况下时间复杂度为 $O(1)$ ，最差情况下为 $O(\log n)$ ，这取决于要查找的数据是否在根节点或叶节点上。
- **B+树**：查询效率稳定为 $O(\log n)$ ，因为所

1.为什么选择b+树而不是b树当索引数据结构?

B+树：非叶节点只包含键，不包含实际数据，所有数据项都存储在叶子节点

1. **IO次数较少**：B+树的非叶节点只包含键，而不包含真实数据，因此每个节点能指向的子节点个数比B树多很多，即阶数 m 更大。这样，**B+树的高度更低，访问时所需的IO次数更少**
2. **查询性能稳定**：由于B+树的所有叶子节点在同一层，并且通过指针连接成链表，这意味着每次查找操作的路径长度都相同，提供了稳定的查找效率。
3. **范围查询高效**：B+树的叶子节点之间的链表结构使得范围查询更加高效。可以顺序地访问叶子节点来快速检索一定范围内的所有记录。

2.b+树索引的特点

B+树索引是一种数据库中常用的索引结构，它是B-Tree的改进版，被广泛应用于数据库和文件系统中。

B+树索引的特点如下：

1. **所有数据记录都是存储在叶子节点上**：这一特性使得B+树能够保持数据的有序性，并且便于范围查找和顺序访问。
2. **非叶子节点仅存储关键字信息（即索引）**：这样做可以减少每个非叶子节点的大小，从而降低树的高度，提高查询效率。
3. **叶子节点之间通过指针连接**：这种设计允许对数据进行高效的顺序访问，对于范围查询特别有用。

4. **平衡性**：与B-Tree一样，B+树是一种平衡多叉树，这意味着树的所有分支都保持大致相同的高度，这有助于保证查询效率的稳定性。
5. **支持动态插入和删除**：B+树的结构允许在不影响整体树结构的情况下，对数据进行动态的插入和删除操作。
6. **适合磁盘存储**：由于磁盘的IO操作相对较慢，B+树的结构减少了必要的磁盘读取次数，因为它能够让查询路径更加紧凑。
7. **支持左闭右开区间搜索**：B+树的搜索采用左闭右开区间，这意味着搜索某个关键字时，会包含等于该关键字的最低值的记录，这对于支持自增ID等场景非常有利。

综上所述，B+树索引因其高效的查询性能、良好的数据有序性和适宜磁盘存储的特性，在数据库系统中得到了广泛的应用。它不仅适用于等值查询，也非常适合处理范围查询和顺序访问的场景。

3.b+树为什么矮胖？

B+树：非叶子节点只包含键，不包含实际数据，所有数据项都存储在叶子节点

1. **节点存储信息多**：B+树的非叶子节点可以存储更多的键值对，这意味着每个节点可以指向更多的子节点，从而增加了树的分支数。所以树的每一层都可以存储更多的数据，减少了整体的层数。
2. **减少磁盘I/O操作**：在数据库系统中，数据的查找通常涉及到磁盘的读取操作。B+树的“矮胖”特性意味着在进行数据查找时，需要遍历的树的层数更少，这样可以减少磁盘的I/O操作次数，提高数据检索的效率。
3. **优化范围查询**：B+树的叶子节点通过指针相连，形成了一个有序链表。这个结构特别适合进行范围查询，因为可以通过遍历叶子节点的链表来快速访问一定范围内的所有记录，而不需要额外的查找操作。

4.哈希索引

哈希索引是基于哈希表实现的，对于每一行数据，存储引擎会对索引列进行哈希计算得到哈希码，并且哈希算法要尽量保证不同的列值计算出的哈希码值是不同的，将哈希码的值作为哈希表的key值，将指向数据行的指针作为哈希表的value值。这样查找一个数据的时间复杂度就是 $O(1)$ ，一般多用于精确查找。

5.什么情况下需要建索引？

1. 经常用于查询的字段
2. 经常用于连接的字段建立索引，可以加快连接的速度
3. 经常需要排序的字段建立索引，因为索引已经排好序，可以加快排序查询速度

6.什么情况下不建索引？

1. `where` 条件中用不到的字段不适合建立索引
2. 表记录较少。比如只有几百条数据，没必要加索引。
3. 需要经常增删改。需要评估是否适合加索引
4. **参与列计算的列**不适合建索引
5. **区分度不高**，数据重复且分布比较均匀的的字段不适合建立索引，如性别，只有男/女/未知三个值。加了索引，查询效率也不会提高。

7.索引类型

0.索引类型

按照底层存储方式角度划分：

- 聚簇索引（聚集索引）：索引结构和数据一起存放的索引，InnoDB 中的主键索引就属于聚簇索引。
- 非聚簇索引（非聚集索引）：索引结构和数据分开存放的索引，二级索引(辅助索引)就属于非聚簇索引。MySQL 的 MyISAM 引擎，不管主键还是非主键，使用的都是非聚簇索引。

按照应用维度划分：

- 主键索引：加速查询 + 列值唯一（不可以有 NULL）+ 表中只有一个。
- 普通索引：仅加速查询。
- 唯一索引：加速查询 + 列值唯一（可以有 NULL）。
- 覆盖索引：一个索引包含（或者说覆盖）所有需要查询的字段的价值。
- 联合索引：多列值组成一个索引，专门用于组合搜索，其效率大于索引合并。
- 全文索引：对文本的内容进行分词，进行搜索。目前只有 `CHAR`、`VARCHAR`，`TEXT` 列上可以创建全文索引。一般不会使用，效率较低，通常使用搜索引擎如 ElasticSearch 代替。

0.为什么联合索引比多个单列索引效率更高

在数据库中，索引是用来提高查询速度的数据结构。当一个表有多个列时，可以为这些列创建单独的索引（单列索引），也可以为这些列一起创建一个组合索引（联合索引）。通常情况下，联合索引比多个单列索引更高效，原因如下：

1. 减少I/O操作：

- 当使用联合索引时，一次查询可以利用同一个索引来定位所需的行，减少了磁盘或存储设备上的读取次数。
- 单列索引需要多次查找才能获取所有需要的信息，这意味着更多的I/O操作。

2. 减少排序和合并操作：

- 如果查询涉及多个条件，并且这些条件分布在不同的单列索引上，那么数据库可能需从每个索引中检索数据，然后进行排序和合并以得到最终结果。
- 联合索引包含所有相关的列，因此可以直接定位到符合条件的记录，避免了额外的排序和合并步骤。

3. 更少的重复数据：

- 多个单列索引可能会导致数据的冗余存储，因为每个索引都需要存储被索引列的值以及指向实际数据行的指针。
- 联合索引只存储一次相关数据，从而节省了存储空间。

4. 更有效的范围查询：

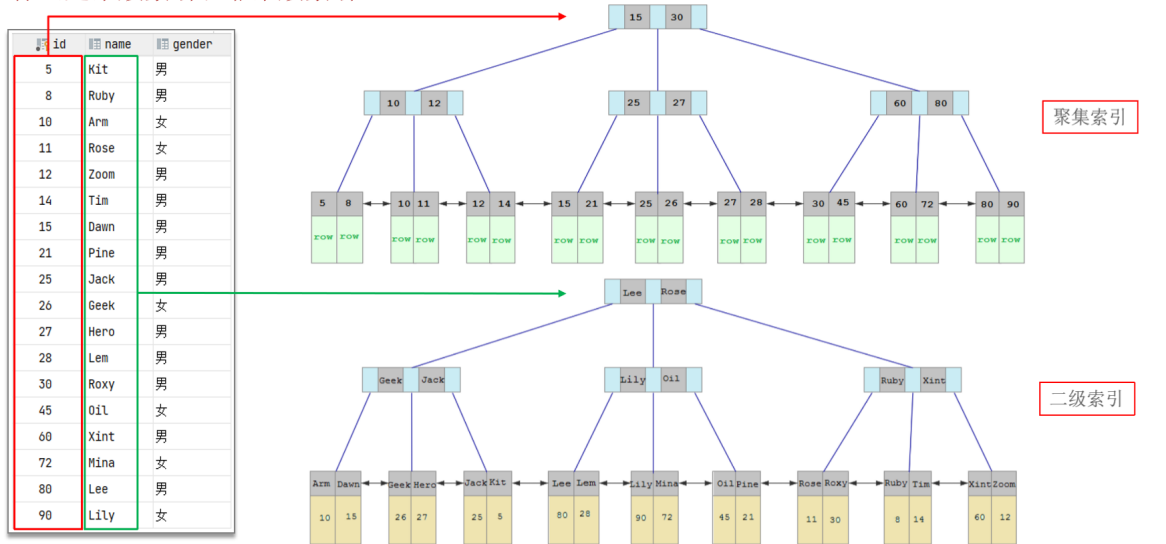
- 如果查询涉及到范围查询（例如 `WHERE column1 BETWEEN value1 AND value2`），联合索引可以有效地处理这种情况，因为它可以在索引树中直接遍历范围。
- 单列索引在这种情况下可能需要额外的操作来确定范围内的行。

1.聚簇索引和非聚簇索引(二级索引)

聚簇索引主要是指数据与索引放到一块，在聚集索引中，叶子节点直接包含了完整的数据行。这意味着如果你查找某个特定的索引值，你将直接访问到对应的数据行。由于这个特性，一个表只能有一个聚集索引，因为数据行只能在一个地方。在InnoDB引擎中，主键索引就是聚集索引。

非聚簇索引值又叫二级索引，数据与索引分开存储。叶子节点不包含完整的数据行，而是包含指向相应数据行的主键值。当你使用二级索引来查找数据时，你需要先查找索引，然后根据得到的主键值再去查找聚集索引中的行数据。一个表可以有多个二级索引。

什么是聚簇索引和非聚簇索引？



回表查询

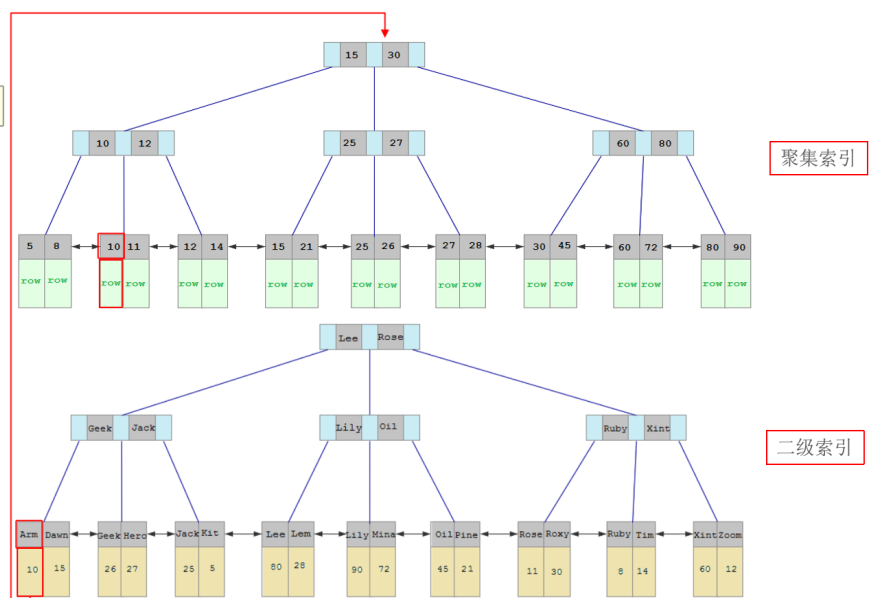
回表的意思就是通过二级索引找到对应的主键值，然后再通过主键值找到聚集索引中所对应的整行数据，这个过程就是回表。

如果按照二级索引查询数据的时候，返回的列中没有创建索引，有可能会触发回表查询，尽量避免使用 `select *`，尽量在返回的列中都包含添加索引的字段

回表查询

```
select * from user where name = 'Arm';
```

回表查询



2. 覆盖索引和联合索引

覆盖索引：如果一个索引包含（或者说覆盖）所有需要查询的字段，我们就称之为 **覆盖索引**，而无需回表查询。

联合索引：使用表中的多个字段创建索引，就是 **联合索引**，也叫 **组合索引** 或 **复合索引**。

以 `score` 和 `name` 两个字段建立联合索引：

```
ALTER TABLE `cus_order` ADD INDEX id_score_name(score, name);
```


最左前缀匹配原则

最左前缀匹配原则指的是，在使用联合索引时，MySQL 会根据联合索引中的字段顺序，从左到右依次到查询条件中去匹配，如果查询条件中存在与联合索引中最左侧字段相匹配的字段，则就会使用该字段过滤一批数据，直至联合索引中全部字段匹配完成，或者在执行过程中遇到范围查询（如 >、<）才会停止匹配，但是对于 >=、<=、BETWEEN、like 前缀匹配的范围查询，并不会停止匹配。所以，我们在使用联合索引时，可以将区分度高的字段放在最左边，这也可以过滤更多数据。

例如，如果按(col1, col2, col3)的顺序创建了联合索引，并且查询中有 col1 = 1 AND col2 > 2 AND col3 = 3 这样的条件，那么只有 col1 能够利用到索引，col2 和 col3 由于 col2 的范围查询，无法利用后续的索引。这是因为 col2 > 2 的范围查询打断了索引的连续匹配过程，因为在 col2 的范围查询之后，col3 字段的值是无序的，无法进一步缩小扫描范围。

8.索引的设计/创建原则？

- 对于经常作为查询条件的字段，应该建立索引，以提高查询速度
- 为经常需要排序、分组和联合操作的字段建立索引
- 索引列的**区分度越高**，索引的效果越好。比如使用性别这种区分度很低的列作为索引，效果就会很差。
- 避免给"大字段"建立索引。尽量使用数据量小的字段作为索引。因为 MySQL 在维护索引的时候是会将字段值一起维护的，那这样必然会导致索引占用更多的空间，另外在排序的时候需要花费更多的时间去对比。
- 尽量使用**短索引**，对于较长的字符串进行索引时应该指定一个较短的前缀长度，因为较小的索引涉及到的磁盘 I/O 较少，查询速度更快。
- 索引不是越多越好，每个索引都需要额外的物理空间，维护也需要花费时间。
- 频繁增删改的字段不要建立索引。假设某个字段频繁修改，那就意味着需要频繁的重建索引，这必然影响 MySQL 的性能
- 利用**最左前缀原则**。

9.什么情况索引会失效

SQL索引失效可能会导致查询速度变慢，因为数据库需要执行全表扫描而不是使用索引

导致索引失效的情况：

- 以%开头的like查询如 %abc，无法使用索引
- 查询条件使用 or 连接，也会导致索引失效：例如，`select * from student_info where student_name='Helen' or student_age > 15`，这种情况下索引不会起作用。原因是数据库无法从OR条件的一个范围查询中获益，只能通过扫描表的方式来满足查询需求。
- 在进行范围查询（如 BETWEEN、>、<、>=、<=）时会失效
- 索引在使用的时候没有遵循**最左匹配法则**(联合索引的匹配从where的最左边的字段开始，只有匹配成功才能继续匹配到右边的下一个字段。)
- 在添加索引的字段上进行了运算操作：例如，若索引列是年龄（age），正常的查询 `select * from student_info where student_age = 21` 会利用索引，但若进行计算 `select * from student_info where student_age + 1 = 21`，则索引失效。这是因为数据库无法直接通过计算后的值去匹配索引。
- 当操作符左右两边的数据类型不一致时，会发生**隐式转换**让索引失效。
- 对索引列进行函数操作：如果有一个基于 enrollment_date 的索引，正常的日期等值查询可以利用索引，而一旦查询中使用了日期函数，如 `YEAR(enrollment_date) = 2022`，索引就会失效。这是因为函数操作改变了索引列的原值，数据库无法直接利用索引进行快速检索。

通常情况下，想要判断出这条sql是否有索引失效的情况，可以使用explain执行计划来分析

为什么范围查询索引会失效

- 当使用范围条件（如 `>`、`<`、`BETWEEN`）进行查询时，如果查询条件中的列是复合索引的一部分，而不是最左侧的列，那么只有该列之前的列才会被用于索引查找，该列及之后的列将不会使用索引。这是因为数据库的索引通常是按照最左前缀原则组织的，即只有从索引的最左侧开始连续的列才能形成有效的索引键。
- 当使用范围条件进行查询时，如使用大于（`>`）或小于（`<`）操作符，会导致索引列的部分数据无序。例如，如果查询条件是 `WHERE c > 10`，那么在 `c` 列上的所有值大于10的行都符合条件，这意味着索引在 `c` 列上的有序性被破坏了，因此无法有效利用索引来快速定位数据。

模糊查询语句中如何使用索引？

在MySQL中模糊查询 `mobile like '%8765'`，这种情况是不能使用 `mobile` 上的索引的，那么如果需要根据手机号码后四位进行模糊查询，可以用一下方法进行改造。

我们可以加入冗余列（MySQL5.7之后加入了虚拟列，使用虚拟列更合适，思路相同），比如 `mobile_reverse`，内部存储为 `mobile` 的倒叙文本，如 `mobile` 为17312345678，那么 `mobile_reverse` 存储 87654321371，为 `mobile_reverse` 列建立索引，查询中使用语句 `mobile_reverse like reverse('%5678')` 即可。

`reverse` 是MySQL中的反转函数，这条语句相当于 `mobile_reverse like '8765%'`，这种语句是可以使用索引的。

10.索引合并index merge

1.前言：单表查询MySQL只能利用一个索引

因为一般情况下，单表查询MySQL只能利用一个索引，比如下面这个查询，假设`id`是主键，`a`和`b`分别创建了索引，别天真的以为 `idx_a` 和 `idx_b` 都能发挥作用，其实不是的。

```
SELECT id,a,b FROM T WHERE a>100 AND b>200;
```

因为 `idx_a` 索引只存储了列`a`和`id`的值，无法判断 `b>200` 条件是否成立，所以只能拿着`id`去回表查询。同样 `idx_b` 索引只存储了列`b`和`id`的值，无法判断 `a>100` 条件是否成立，也只能拿着`id`去回表查询。可以看到，最大的开销其实是回表操作，通过二级索引匹配到的数据越少，回表的开销也就越低。所以理论上来说，`a>100` 和 `b>200` 分别符合这两个条件的记录数越少，MySQL就会使用哪个索引。MySQL是如何判断符合这些条件的记录数量的呢？不也得老老实实的扫描全表吗？MySQL采用预估的方式，通过表的统计数据或访问表中少量的数据来进行预估，并分别计算使用这两个索引进行查询各自的成本是多少，最终选择执行成本更低的索引方案。

我们假设最终MySQL使用 `idx_a` 索引，那么这个查询过程其实是这样的：

1. InnoDB从 `idx_a` B+树中获取到第一条 `a>100` 的记录，拿记录里的`id`值回表查询。
2. 回表查询获取到完整的用户记录，判断 `b>200` 是否成立，成立则返回给客户端，否则丢弃该记录。
3. InnoDB继续从 `idx_a` B+树中获取到下一条 `a>100` 的记录，重复前面的过程。

建立了这么多索引，每次查询只使用一个，太可惜了不是嘛。能不能同时利用多个索引来完成查询呢？可以的，但是条件有些严苛，这就是我们今天要介绍的索引合并Index Merge。

2.index merge索引合并

在 MySQL 中，当执行一个查询语句需要使用多个索引时，MySQL 可以使用索引合并(Index Merge)来优化查询性能。具体来说，索引合并是将多个单列索引或多个联合索引合并使用，以满足查询语句的需要。

当使用索引合并时，MySQL 会选择最优的索引组合来执行查询，从而避免了全表扫描和排序操作，提高了查询效率。而对于使用多个单列索引的查询语句，MySQL 也可以使用索引合并来优化查询性能。

如何才能知道我们写的 SQL 语句使用了索引合并呢？通过 EXPLAIN 分析一下就知道了，如果使用了索引合并，对应的 type 列显示的值应该是 index_merge，key 列显示用的到所有索引名称，Extra 列会显示具体使用了哪种类型的索引合并。

索引合并主要有三种类型：交集合并(Intersection Merge)、并集合并(Union Merge)和排序并集合并(Sort-Union Merge)。每种类型适用于不同查询场景，并可根据具体的查询条件和所需结果进行选择。

- 交集合并(Intersection Merge) 适用于查询需要满足多个条件(用AND连接)并且每个条件都可以使用不同索引的情况。在这种情况下，MySQL 会分别扫描这些索引，然后取结果的交集。例如，假设有一个用户表 users，包含字段 id (主键)、age、city 和 income，且这些字段上都有索引。如果查询语句是：

```
SELECT * FROM users WHERE age = 30 AND city = 'New York';
```

在这个查询中，age 和 city 是两个独立的条件，每个条件都可以使用不同的索引。MySQL 可能会决定使用交集合并策略，分别扫描 age 索引和 city 索引，然后取结果的交集，最终返回满足 age = 30 且 city = 'New York' 的用户记录。

- 并集合并(Union Merge) 则适用于查询只需满足多个条件中的任意一个(用OR连接)的情况。在这种情况下，MySQL 会分别扫描这些索引，然后取结果的并集。以上面的 users 表为例，如果查询语句如下：

```
sql复制代码SELECT * FROM users WHERE age = 30 OR city = 'Los Angeles';
```

在这个查询中，只要满足 age = 30 或 city = 'Los Angeles' 中的任意一个条件，记录就会被选中。MySQL 可能会使用并集合并策略，分别扫描 age 索引和 city 索引，然后合并结果集，返回满足任一条件的用户记录。

- 排序并集合并(Sort-Union Merge) 是一种特殊形式，主要用于需要在结果排序的场景中。尽管它不是直接支持所有情况，但基本思路是分别扫描索引，然后将结果合并并排序。考虑到实际执行计划可能有所不同，具体操作时需结合 EXPLAIN 命令查看查询的执行计划，根据实际情况进行调整和优化。

11.倒排索引

倒排索引是一种数据库索引技术，用于存储从内容到文档的映射，以优化全文搜索的效率。倒排索引，是实现“单词-文档矩阵”的一种具体存储形式，它能快速地根据单词获取包含这个单词的文档列表

倒排索引的核心思想与传统的文件到关键词的索引方式相反。在传统的正向索引中，索引是按照“文档→关键词”的方式来组织的，这意味着要查找包含特定关键词的所有文档，需要检查每个文档并提取关键词。这种方法在处理大量文档时效率较低。而倒排索引则采用“关键词→文档”的方式，即它首先记录了每个关键词出现在哪些文档中，这样当用户搜索某个关键词时，可以直接找到包含该关键词的所有文档，极大地提高了搜索效率。

以下是倒排索引的几个关键特点：

1. **关键词列表**：对于每个关键词，倒排索引会列出一个包含该关键词的所有文档ID的列表。
2. **快速检索**：通过关键词可以快速定位到相关文档，无需遍历整个文档集合。
3. **支持全文搜索**：倒排索引能够有效地支持全文搜索，是现代搜索引擎和信息检索系统中不可或缺的技术。
4. **空间效率**：虽然倒排索引可能需要较多的存储空间来维护关键词到文档的映射，但它为检索速度的提升提供了空间换时间的优化。

总的来说，倒排索引是信息检索领域的基础技术之一，它的应用不仅仅局限于搜索引擎，还包括任何需要快速文本检索的场景。

12.什么是索引下推？

索引下推是MySQL 5.6引入的一项优化技术，它主要用于提高查询效率。**索引下推的核心思想是在服务层进行索引扫描时，将部分WHERE条件下推到存储引擎层，据这些过滤条件进行判断，只返回满足条件的记录，从而减少回表查询和服务层与存储引擎层之间的数据交互，提高查询效率。**

在没有索引下推的情况下，MySQL会先在服务层进行索引扫描，然后将满足条件的记录返回给存储引擎层进行进一步的过滤。这个过程涉及到大量的数据交互，可能会导致查询效率降低。

有了索引下推后，MySQL会在服务层进行索引扫描的同时，将过滤条件传递给存储引擎层。存储引擎层在读取记录时，会根据这些过滤条件进行判断，只返回满足条件的记录。这样可以减少服务层与存储引擎层之间的数据交互，提高查询效率。

假设有一个用户表 `tuser`，并创建了一个联合索引 `(name, age)`。如果查询需求是检索名字第一个字是“张”且年龄为10岁的所有用户，那么在没有使用ICP的情况下，存储引擎会根据联合索引找到 `name like '张%'` 的主键id，逐一进行回表扫描并检索完整记录，然后再根据 `age=10` 的条件进行筛选。而使用ICP后，存储引擎会在联合索引中直接按照 `age=10` 进行过滤，大大减少了回表扫描的次数。

总之，索引下推是一种优化技术，通过将过滤条件下推到存储引擎层，减少数据交互，提高查询效率。

13.大表如何添加索引

如果一张表数据量级是千万级别以上的，那么，如何给这张表添加索引？

我们需要知道一点，**给表添加索引的时候，是会对表加锁的**。如果不谨慎操作，有可能出现生产事故的。可以参考以下方法：

1. 先创建一张跟原表 `A` 数据结构相同的新表 `B`。
2. 在新表 `B` 添加需要加上的新索引。
3. 把原表 `A` 数据导到新表 `B`
4. `rename` 新表 `B` 为原表的表名 `A`，原表 `A` 换别的表名；

3.事务相关

1.事务特性ACID

事务特性ACID：**原子性**（Atomicity）、**一致性**（Consistency）、**隔离性**（Isolation）、**持久性**（Durability）。

- **原子性**是指事务包含的所有操作要么全部成功，要么全部失败回滚。
- **一致性**是指一个事务执行之前和执行之后都必须处于一致性状态。比如a与b账户共有1000块，两人之间转账之后无论成功还是失败，它们的账户总和还是1000。

- **隔离性。**事务的隔离性要求**该事务提交前对其他事务都不可见**，这通常使用锁来实现。跟隔离级别相关，如 `read committed`，一个事务只能读到已经提交的修改。**追求的是并发情形下事务之间互不干扰**
- **持久性**是指一个事务一旦被提交了，那么对数据库中的数据的改变就是永久性的，即便是在数据库系统遇到故障的情况下也不会丢失提交事务的操作。

2. MySQL的ACID特性分别是怎么实现的？

原子性(undo log)

在MySQL中，事务的原子性是通过**undo log（归滚日志）**来保证的。

undo log属于逻辑日志，它记录的是sql执行相关的信息。当发生回滚时，InnoDB会根据undo log的内容做与之前相反的工作。对于insert，回滚时会执行delete。对于delete，回滚时会执行insert。对于update，回滚时则会执行相反的update，把数据改回去。

一致性

可以说，**一致性是事务追求的最终目标**。前面提到的原子性、持久性和隔离性，都是为了保证数据库状态的一致性。此外，除了数据库层面的保障，一致性的实现也需要应用层面进行保障。实现一致性的措施包括：

保证原子性、持久性和隔离性，如果这些特性无法保证，事务的一致性也无法保证。

数据库本身提供保障，例如不允许向整形列插入字符串值、字符串长度不能超过列的限制等。

应用层面进行保障，例如如果转账操作只扣除转账者的余额，而没有增加接收者的余额，无论数据库实现的多么完美，也无法保证状态的一致。

隔离性

- **锁机制：**MySQL使用不同的锁来保证事务的隔离性，包括行锁、表锁以及元数据锁等。这些锁可以防止多个事务同时修改同一数据集，避免数据的不一致性、脏读和不可重复读等问题。
 - 隔离性要求同一时刻只能有一个事务对数据进行写操作，InnoDB通过锁机制来保证这一点。
锁机制的基本原理可以概括为：事务在修改数据之前，需要先获得相应的锁。获得锁之后，事务便可以修改数据。该事务操作期间，这部分数据是锁定的，其他事务如果需要修改数据，需要等待当前事务提交或回滚后释放锁。
按照粒度，锁可以分为表锁、行锁以及其他位于二者之间的锁。表锁在操作数据时会锁定整张表，并发性能较差。行锁则只锁定需要操作的数据，并发性能好。但是由于加锁本身需要消耗资源，因此在锁定数据较多情况下使用表锁可以节省大量资源。MySQL中不同的存储引擎支持的锁是不一样的，例如**MyIsam只支持表锁，而InnoDB同时支持表锁和行锁**，且出于性能考虑，绝大多数情况下使用的都是行锁
- **MVCC的作用：**除了锁机制外，InnoDB存储引擎还采用MVCC来实现事务的隔离性。MVCC通过对数据版本的控制，保证了不同事务之间的数据访问不会互相干扰，并针对不同的隔离级别提供了相应的视图。
 - InnoDB默认的隔离级别是RR（REPEATABLE READ），RR解决脏读、不可重复读、幻读等问题，使用的是MVCC。MVCC全称Multi-Version Concurrency Control，即多版本的并发控制协议。它最大的优

点是读不加锁，因此读写不冲突，并发性能好。InnoDB实现MVCC，多个版本的数据可以共存，主要基

于以下技术及数据结构：

\1. 隐藏列：InnoDB中每行数据都有隐藏列，隐藏列中包含了本行数据的事务id、指向undo log的指针等。

\2. 基于undo log的版本链：每行数据的隐藏列中包含了指向undo log的指针，而每条undo log也会

指向更早版本的undo log，从而形成一条版本链。

\3. ReadView：通过隐藏列和版本链，MySQL可以将数据恢复到指定版本。但是具体要恢复到哪个版本，则需要根据ReadView来确定。所谓ReadView，是指事务（记做事务A）在某一时刻给整个事务系统（trx_sys）打快照，之后再进行读操作时，会将读取到的数据中的事务id与trx_sys快照比

较，从而判断数据对该ReadView是否可见，即对事务A是否可见。

较，从而判断数据对该ReadView是否可见，即对事务A是否可见。

- **隔离级别**：MySQL支持多种隔离级别（读未提交、读已提交、可重复读和串行化），这些隔离级别通过不同的手段解决脏读、不可重复读和幻读等问题。

持久性(redo log)

D (durability) ，持久性。事务一旦提交，其结果就是永久性的，即使发生宕机等故障，数据库也能将数据恢复。持久性保证的是事务系统的高可靠性，而不是高可用性。

InnoDB作为MySQL的存储引擎，数据是存放在磁盘中的，但如果每次读写数据都需要磁盘IO，效率会很低。为此，InnoDB提供了**缓存(Buffer Pool)**，Buffer Pool中包含了磁盘中部分数据页的映射，作为访问数据库的缓冲。**当从数据库读取数据时，会首先从Buffer Pool中读取，如果Buffer Pool中没有，则从磁盘读取后放入Buffer Pool。当向数据库写入数据时，会首先写入Buffer Pool，Buffer Pool中修改的数据会定期刷新到磁盘中（这一过程称为刷脏）。**

Buffer Pool的使用大大提高了读写数据的效率，但是也带了新的问题：如果MySQL宕机，而此时Buffer Pool中修改的数据还没有刷新到磁盘，就会导致数据的丢失，事务的持久性无法保证。

于是，redo log被引入来解决这个问题。**当数据修改时，除了修改Buffer Pool中的数据，还会在redo log记录这次操作。当事务提交时，会调用fsync接口对redo log进行刷盘。如果MySQL宕机，重启时可以读取redo log中的数据，对数据库进行恢复。**redo log采用的是WAL（Write-ahead logging，预写式日志），所有修改先写入redo log日志，再更新到Buffer Pool，保证了数据不会因MySQL宕机而丢失，从而满足了持久性要求

为了确保事务的持久性，MySQL使用redo log（重做日志）。事务对数据的修改先被记录在redo log中，之后在适当的时机将这些改动刷新到磁盘上。这样即便发生宕机，也可以通过redo log恢复数据。

为了确保事务的持久性，MySQL使用redo log（重做日志）。事务对数据的修改先被记录在redo log中，之后在适当的时机将这些改动刷新到磁盘上。这样即便发生宕机，也可以通过redo log恢复数据。

2.并发事务问题(脏读，幻读等)及4种隔离级别

1.脏读、不可重复读、幻读

脏读、幻读和不可重复读是数据库并发事务处理中的三种常见问题。

1. 脏读：在一个事务中读取到了另一个事务还未提交的数据，如果这个事务最终回滚，那么之前读取到的数据就是“脏数据”，这种现象称为脏读。
2. 不可重复读：在一个事务内多次读取同一数据，但由于其他事务的提交导致数据发生改变，造成每次读取的结果不同，这种现象称为不可重复读。这是由于在查询间隔，另一个事务修改了数据并提

交了。

3. 幻读：在同一事务中，两次查询某个范围内的记录时，第二次查询返回第一次查询中没有的额外记录，这是因为另外一个事务又在该范围内插入了新的记录。

不可重复读和脏读的区别是，脏读是某一事务读取了另一个事务未提交的脏数据，而不可重复读则是读取了前一事务已经提交的数据。

不可重复读和幻读的区别是，不可重复读关注的是对已有数据的修改，而幻读关注的是对数据范围的新增数据。

2.4种事务隔离级别

哪4种隔离级别？

事务隔离就是为了解决上面提到的脏读、不可重复读、幻读这几个问题。

MySQL数据库为我们提供的四种隔离级别：

- **Serializable(串行化)**：这种隔离级别模拟了事务串行执行，就像一个接一个执行一样，。通过强制事务排序，使之不可能相互冲突，从而解决幻读问题。但会显著降低并发处理能力
- **Repeatable read(可重复读)**：MySQL的默认事务隔离级别，**确保如果在一个事务内多次读取同一行数据，结果总是一致**。防止了脏读和不可重复读，但是幻读仍然可能发生。为了实现这个隔离级别，mysql使用了一种称为多版本并发控制（mvcc）的技术。
- **Read committed(读已提交)**：只有在事务提交后，其他事务才能看到其所做的修改，从而避免了脏读。然而，它仍然允许不可重复读（在一个事务中，两次相同的查询可能得到不同的结果）和幻读（在一个事务中执行查询，另一个事务新增了记录并提交，第一个事务再次查询时会发现有额外的记录）
- **Read uncommitted(读未提交)**：允许事务读取尚未提交的其他事务修改的数据，因此可能发生脏读、不可重复读和幻读。

隔离级别	脏读	不可重复读	幻读
读未提交（Read uncommitted）	V	V	V
读已提交（Read committed）	X	V	V
可重复读（Repeatable read）	X	X	V
可串行化（Serializable）	X	X	X

4种隔离级别的实现机制

这四种隔离级别的实现机制如下：

- READ UNCOMMITTED & READ COMMITTED：
通过Record Lock算法实现了行锁，但READ UNCOMMITTED允许读取未提交数据，所以存在脏读问题。而READ COMMITTED允许读取提交数据，所以不存在脏读问题，但存在不可重复读问题。
- REPEATABLE READ：
使用Next-Key Lock算法实现了行锁，并且不允许读取已提交的数据，所以解决了不可重复读的问题。另外，该算法包含了间隙锁，会锁定一个范围，因此也解决了幻读的问题。
- SERIALIZABLE：
对每个SELECT语句后自动加上LOCK IN SHARE MODE，即为每个读取操作加一个共享锁。因此在这个事务隔离级别下，读占用了锁，对一致性的非锁定读不再予以支持。

可重复读隔离级别怎么解决不可重复读问题？

在可重复读隔离级别下，InnoDB使用**多版本并发控制（MVCC）**和**行级锁**来实现这一行为。当一个事务读取数据时，它会读取符合事务开始时快照的数据版本，而不会看到其他事务对数据的更改。如果一个事务需要写入数据，它会对数据加锁，防止其他事务同时修改相同的数据，但其他事务仍然可以读取旧的数据版本。

需要注意的是，在可重复读隔离级别下，InnoDB会存在幻读现象（Phantom Reads），即同一个事务在两次执行相同的范围查询时，可能返回不同数量的结果集，因为其他事务在该范围内插入了新的行。但是，InnoDB通过**Next-Key Locks**机制可以避免非唯一索引上的幻读问题。如果要完全避免幻读，可以将事务隔离级别提升到串行化（Serializable）。不过，这通常会导致更多的锁竞争和较低的并发性能。

场景题

在可重复读这个隔离级别下，有两个事务AB，对一个金额数据进行更改，A事务每次加10，B事务每次加20，A事务执行1秒，B事务执行10秒，B事务执行到第3秒的时候A事务执行，请问最后金额的值是多少？

3.MySQL 的隔离级别怎么实现的？

MySQL 的隔离级别基于锁和 MVCC 机制共同实现的。

可串行化 隔离级别是通过锁来实现的，读已提交 和可重复读 隔离级别是基于 MVCC 实现的。不过，可串行化 之外的其他隔离级别可能也需要用到锁机制，就比如 REPEATABLE-READ 在当前读情况下需要使用加锁读来保证不会出现幻读。

MySQL 的隔离级别是通过锁和多版本并发控制（MVCC）来实现的。

1. 锁：MySQL 提供了多种类型的锁，如共享锁（S）、排他锁（X）、意向锁等。这些锁可以用于实现不同级别的隔离性。例如，在可重复读（REPEATABLE READ）隔离级别下，MySQL 使用行级锁来确保在同一事务中多次读取同一行数据时，数据的一致性。
2. 多版本并发控制（MVCC）：MVCC 是一种并发控制技术，它允许多个事务同时访问数据库中的同一份数据，而不会互相干扰。在 MySQL 中，每个事务都有一个唯一的事务ID，每个被修改的数据行都有一个隐藏的版本号。当一个事务尝试读取或修改数据时，它会检查当前事务ID与数据行的最新版本号是否匹配。如果匹配，说明该事务可以看到该版本的数据；如果不匹配，说明该事务需要等待其他事务完成并提交它们的更改。

通过结合锁和 MVCC，MySQL 可以实现不同的隔离级别，以满足不同的应用场景需求。例如，在读已提交（READ COMMITTED）隔离级别下，只允许读取已经提交的数据；而在可重复读（REPEATABLE READ）隔离级别下，同一个事务内的多次读取操作会看到相同的数据。

4.事务可以嵌套吗？

可以，因为嵌套事务也是众多事务分类中的一种，它是一个层次结构框架。有一个顶层事务控制着各个层次的事务，顶层事务之下嵌套的事务被称为子事务，它控制每一个局部的变换。

需要注意的是，MySQL数据库不支持嵌套事务。

5.怎么实现回滚？

在MySQL中，事务的原子性是通过**undo log（归滚日志）**来保证的。

undo log属于逻辑日志，它记录的是sql执行相关的信息。当发生回滚时，InnoDB会根据undo log的内容做与之前相反的工作。对于insert，回滚时会执行delete。对于delete，回滚时会执行insert。对于update，回滚时则会执行相反的update，把数据改回去。

4.三大日志

1.MySQL 的 change buffer 是什么？

主要用于暂存那些未被加载进内存的非主键索引页上的更改操作。当进行插入、更新或删除等DML（数据操作语言）操作时，如果涉及到的非主键索引页不在内存中，这些操作会被缓存在Change Buffer中，而不是立即进行磁盘IO操作，从而减少了即时的磁盘访问，提升了操作效率。

- 当需要更新一个数据页时，如果数据页在内存中就直接更新；而如果这个数据页还没有在内存中的话，在不影响数据一致性的前提下，InnoDB 会将这些更新操作缓存在 change buffer 中。
- 这样就不需要从磁盘中读入这个数据页了，在下次查询需要访问这个数据页的时候，将数据页读入内存，然后执行 change buffer 中与这个页有关的操作。通过这种方式就能保证这个数据逻辑的正确性。
- 注意唯一索引的更新就不能使用 change buffer，实际上也只有普通索引可以使用。
- 适用场景：
 - 对于写多读少的业务来说，页面在写完以后马上被访问到的概率比较小，此时 change buffer 的使用效果最好。这种业务模型常见的就是账单类、日志类的系统。
 - 反过来，假设一个业务的更新模式是写入之后马上会做查询，那么即使满足了条件，将更新先记录在 change buffer，但之后由于马上要访问这个数据页，会立即触发 merge 过程。这样随机访问 IO 的次数不会减少，反而增加了 change buffer 的维护代价

2.bin log/redo log/undo log日志

0.它们是什么？

`bin log`（二进制日志）和 `redo log`（重做日志）和 `undo log`（回滚日志）。

bin log二进制日志

Bin Log是MySQL的二进制日志，记录了所有更改数据或可能更改数据的SQL语句，并以二进制格式保存在磁盘上

binlog主要用于数据备份和主从同步。

redo log重做日志

redo log是重做日志，记录的是数据修改之后的值，不管事务是否提交都会记录下来。它的主要作用是保证服务崩溃后，仍能把事务中变更的数据持久化到磁盘上。

redo log保证了事务的持久性，比如断电了，InnoDB存储引擎会使用redo log恢复到断电前的时刻，以此来保证数据的完整性。

undo log回滚日志

Undo Log是MySQL的回滚日志，它用于在事务回滚或者数据库崩溃时撤销已提交的事务对数据库的修改。Undo Log记录了**记录了事务对数据库所做的更改的相反操作**。例如，如果事务执行了一条INSERT语句，那么Undo Log会记录一个DELETE操作；若执行UPDATE，则记录一个相反的UPDATE操作。，以便在需要回滚时可以恢复数据。

undo log则负责事务的回滚和实现MVCC，保证了事务的原子性和一致性

1.undo log 与redo log的区别

Undo Log和Redo Log是InnoDB存储引擎中实现事务持久性和原子性的关键机制，但它们的功能和应用场景不同。下面将从多个维度详细对比这两种日志：

1. 基本概念

- **Undo Log**：记录事务对数据所做的修改前的状态，用于事务回滚和MVCC（多版本并发控制）。
- **Redo Log**：记录数据页的物理修改操作，主要用于系统崩溃后的恢复和保证事务的持久性。

2. 应用场景

- **Undo Log**：用于事务回滚时恢复数据到原始状态，支持MVCC以提供一致性读视图，避免锁竞争。
- **Redo Log**：在数据库崩溃后通过重放日志保证已提交事务的修改不会丢失，采用WAL策略提升写入性能。

3. 存储内容

- **Undo Log**：记录数据的逻辑变化，如INSERT操作会记录相反的DELETE操作信息。
- **Redo Log**：记录物理数据页面的修改信息，如数据页的变更。

2.bin log和redo log有什么区别？

下面将从多个维度详细对比Bin Log和Redo Log：

1. 基本概念

- **Bin Log**：记录了对MySQL数据库执行更改的所有操作，包括DDL和DML语句（不包括查询语句），以事件形式记录。
- **Redo Log**：记录数据页的物理修改操作，主要用于系统崩溃后的恢复。

2. 应用场景

- **Bin Log**：用于数据恢复、主从复制以及数据备份。
- **Redo Log**：保证事务的持久性，防止系统崩溃导致的数据丢失，提升写入性能。

3. 记录内容

- **Bin Log**：记录逻辑操作，如SQL语句及其增删改操作的反向信息。
- **Redo Log**：记录物理数据页面的修改信息。

3.redo log 的执行流程？

Redo Log（重做日志）是MySQL数据库中的一种日志文件，用于记录数据库中每个事务的修改操作。它的执行流程如下：

1. 当一个事务开始时，InnoDB存储引擎会为此事务分配一个连续的、且自增的log sequence number (LSN, 日志序列号)。
2. 在事务执行过程中，每当有数据被修改，InnoDB存储引擎就会将这个修改作为一个“redo log record”写入到Redo Log Buffer（重做日志缓冲区）中。这个记录包含了修改的类型（插入、更新或删除）、修改的表空间ID、修改的数据页号、修改的行信息等。
3. 当事务提交时，InnoDB存储引擎会将Redo Log Buffer中的redo log record写入到Redo Log文件中。这个过程是异步进行的，也就是说，即使redo log record还没有被写入到磁盘上的Redo Log文件中，事务也可以正常提交。
4. 当系统发生故障时，如服务器宕机或者数据库进程异常退出，InnoDB存储引擎可以通过Redo Log文件中的redo log record来恢复未完成的事务。具体来说，它会扫描所有的Redo Log文件，找到未完成的事务，然后根据redo log record重新执行这些事务的修改操作，从而保证数据的一致性。

5. Redo Log文件的大小是有限的，当Redo Log Buffer中的redo log record被写入到Redo Log文件后，这些redo log record就会被标记为“已提交”。同时，InnoDB存储引擎会定期地将Redo Log文件中的“已提交”的redo log record进行清理，以释放磁盘空间。这个过程被称为“redo log compaction”。

总之，Redo Log的主要作用是记录数据库中每个事务的修改操作，以便在系统发生故障时能够恢复未完成的事务，保证数据的一致性。

4.为什么 redo log 具有 crash-safe 的能力，是 binlog 无法替代的？

第一点：redo log 可确保 innoDB 判断哪些数据已经刷盘，哪些数据还没有

- redo log 和 binlog 有一个很大的区别就是，一个是循环写，一个是追加写。也就是说 redo log 只会记录未刷盘的日志，已经刷入磁盘的数据都会从 redo log 这个有限大小的日志文件里删除。binlog 是追加日志，保存的是全量的日志。
- 当数据库 crash 后，想要恢复**未刷盘但已经写入 redo log 和 binlog 的数据**到内存时，binlog 是无法恢复的。虽然 binlog 拥有全量的日志，但没有一个标志让 innoDB 判断哪些数据已经刷盘，哪些数据还没有。
- 但 redo log 不一样，只要刷入磁盘的数据，都会从 redo log 中抹掉，因为是循环写！数据库重启后，直接把 redo log 中的数据都恢复至内存就可以了。

第二点：如果 redo log 写入失败，说明此次操作失败，事务也不可能提交

- redo log 每次更新操作完成后，就一定会写入日志，如果**写入失败**，说明此次操作失败，事务也不可能提交。
- redo log 内部结构是基于页的，记录了这个页的字段值变化，只要crash后读取redo log进行重放，就可以恢复数据。
- 这就是为什么 redo log 具有 crash-safe 的能力，而 binlog 不具备。

4.存储引擎

1.常见存储引擎

MySQL中常用的四种存储引擎分别是：**MyISAM、InnoDB、MEMORY、ARCHIVE。**

MySQL 当前默认的存储引擎是 InnoDB。并且，所有的存储引擎中只有 InnoDB 是事务性存储引擎，也就是说只有 InnoDB 支持事务。

InnoDB存储引擎

InnoDB是MySQL**默认的事务型存储引擎**，使用最广泛，基于聚簇索引建立的。InnoDB内部做了很多优化，如能够自动在内存中创建自适应hash索引，以加速读操作。

优点：支持事务和崩溃修复能力；引入了行级锁和外键约束。

缺点：占用的数据空间相对较大。

适用场景：需要事务支持，并且有较高的并发读写频率。

2.MyISAM 和 InnoDB 有什么区别？

1. **可移植性、备份及恢复。** MyISAM数据是以文件的形式存储，所以在跨平台的数据转移中会很方便。在备份和恢复时可单独针对某个表进行操作。对于InnoDB，可行的方案是拷贝数据文件、备份 binlog，或者用mysqldump，在数据量达到几十G的时候就相对麻烦了。
2. **是否支持行级锁。** MyISAM 只支持表级锁，用户在操作myisam表时，select，update，delete，insert语句都会给表自动加锁，如果加锁以后的表满足insert并发的情况下，可以在表的尾部插入

新的数据。而InnoDB 支持行级锁和表级锁，默认为行级锁。行锁大幅度提高了多用户并发操作的性能。

3. **是否支持事务。** MyISAM 不提供事务支持。而InnoDB 提供事务支持，具有事务、回滚和崩溃修复能力。
4. **是否支持外键。** MyISAM不支持，而InnoDB支持。
5. **是否支持MVCC。** MyISAM不支持，InnoDB支持。应对高并发事务，MVCC比单纯的加锁更高效。
6. **是否支持聚集索引。** MyISAM不支持聚集索引，InnoDB支持聚集索引。

1.行级锁和表级锁

行级锁和表级锁是数据库中用于控制并发操作的两种不同类型的锁。以下是具体分析：

- **行级锁：**行级锁是细粒度的锁，它允许多个事务同时对同一表的不同行进行操作，只要它们不冲突。这种锁对于高并发环境下的数据库性能是非常有益的，因为它允许更高的并行度。InnoDB存储引擎支持行级锁，并且在默认情况下使用行级锁。
- **表级锁：**表级锁是粗粒度的锁，它会锁定整张表，使得在锁持有期间，其他事务无法对表中的任何数据进行写入或修改。这种锁的开销较小，加锁速度快，但会限制并发性，因为一次只有一个事务可以写入或修改表。MyISAM和MEMORY存储引擎采用表级锁，而InnoDB在特定情况下也会使用表级锁。

总结来说，行级锁提供了更好的并发性能，适用于数据量大且需要高并发访问的场景；而表级锁在操作少量数据时更为高效，但在高并发场景下可能会成为瓶颈。在实际应用中，选择合适的锁策略取决于具体的工作负载和性能需求。

2.行锁有哪几种？

行锁主要有记录锁和间隙锁两种类型。以下是对这两种行锁的详细解释：

- **记录锁 (Record Locks) :**
 1. 记录锁是直接锁定某行记录的锁，通常在执行更新操作时使用。
 2. 它要求被锁定的列必须是主键或唯一索引列，以确保锁定的是单一记录。
 3. 查询语句中的条件必须使用精确匹配（如等号=），而不是范围查询（如大于>、小于<、LIKE等），否则可能退化为其他类型的锁，例如临键锁。
- **间隙锁 (Gap Locks) :**
 1. 间隙锁是锁定一个范围内的索引记录，主要用于防止其他事务插入到这个范围内。
 2. 它基于非唯一索引，当使用范围条件查询时，InnoDB会给符合条件的所有行加上间隙锁。

此外，MySQL还支持多种锁模式，如共享锁（S锁）和排它锁（X锁），以及意向锁等，这些锁模式可以与行锁结合使用，以提供更灵活的并发控制机制。

在实际使用中，行锁的选择和配置取决于具体的应用场景和性能要求。理解不同类型行锁的特点和适用场景，可以帮助数据库管理员或开发者优化数据库的性能，特别是在高并发的环境中。

3.innoDB 数据页结构

一个数据页大致划分七个部分

- File Header：表示页的一些通用信息，占固定的38字节。
- page Header：表示数据页专有信息，占固定的56字节。
- inimum+Supermum：两个虚拟的伪记录，分别表示页中的最小记录和最大记录，占固定的26字节。
- User Records：真正存储我们插入的数据，大小不固定。
- Free Space：页中尚未使用的部分，大小不固定。
- Page Directory：页中某些记录的相对位置，也就是各个槽对应的记录在页面中的地址偏移量。

- File Trailer：用于检验页是否完整，占固定大小 8 字节。

4.InnoDB 为什么要用自增 ID 作为主键？

- 自增主键的插入模式，符合递增插入，每次都是追加操作，不涉及挪动记录，也不会触发叶子节点的分裂。
- 每次插入新的记录就会顺序添加到当前索引节点的后续位置，当一页写满，就会自动开辟一个新的页。
- 而有业务逻辑的字段做主键，不容易保证有序插入，由于每次插入主键的值近似于随机
- 因此每次新纪录都要被插到现有索引页得中间某个位置， 频繁的移动、分页操作造成了大量的碎片，得到了不够紧凑的索引结构，写数据成本较高。

5.Mysql的锁

0.悲观锁 sql怎么写

悲观锁是一种数据库锁定策略，用于在事务处理过程中防止其他事务修改数据。在SQL中，可以使用

`SELECT ... FOR UPDATE` 语句来实现悲观锁。

```
BEGIN; -- 开始事务
SELECT * FROM table_name WHERE id = 1 FOR UPDATE; -- 对id为1的记录加锁
-- 在这里进行数据处理操作
COMMIT; -- 提交事务，释放锁
```

乐观锁sql怎么写？

乐观锁是一种数据库锁定策略，用于在事务处理过程中防止其他事务修改数据。它的基本思想是在更新数据时检查数据是否已被其他事务修改，如果没有被修改则进行更新，否则放弃更新并提示冲突。在SQL中，可以使用 `WHERE` 子句来实现乐观锁。

```
BEGIN; -- 开始事务
-- 假设表中有一个名为version的字段，用于记录版本号
UPDATE table_name SET column1 = 'new_value1', column2 = 'new_value2', version =
version + 1 WHERE id = 1 AND version = expected_version;
-- 如果更新成功，expected_version应该等于原来的version值加1
COMMIT; -- 提交事务
```

行锁和表锁什么时候加？业务场景有哪些？

行锁是在事务处理期间锁定具体的行，而不是整个表。这可以提高并发性能，因为多个事务可以同时操作不同的行。

表锁是在事务处理期间锁定整个表，这会阻止其他事务对表进行任何读写操作。

行锁**使用场景**：

1. **高并发读写操作**：当系统需要支持大量的并发读写操作时，使用行锁可以有效避免阻塞其他不相关的事务。
2. **局部数据更新**：如果事务只需要更新表中的一部分行，那么行锁是一个很好的选择。
3. **利用索引**：行锁只在通过索引访问数据时生效。如果查询没有使用索引，InnoDB可能会退化到使用表锁。
4. **锁定特定行**：当你需要确保在事务过程中某个特定行的数据不会被其他事务更改时，可以使用行锁。

表锁使用场景:

1. **批量数据操作**: 当需要对表中的大量数据进行操作时, 使用表锁可以简化锁管理。
2. **低并发环境**: 如果应用程序的并发度较低, 且大多数操作是读取而非写入, 使用表锁可以简化锁管理并减少资源消耗。
3. **数据备份/恢复**: 在进行全表备份或恢复操作时, 通常需要锁定整个表来确保数据的一致性。

1.行级锁、表级锁和页级锁

按锁粒度分类, 有行级锁、表级锁和页级锁。

1. **行级锁具有最小的锁定粒度, 针对索引字段加的锁, 仅针对表中的一行或几行数据进行锁定。它的优点是能最大限度地提高并发处理能力, 因为不同的事务可以在同一时间内操作表中的不同行。缺点包括开销大、加锁慢, 以及可能出现死锁的风险。适用于多事务需要操作同一表的不同行时使用。**
2. **表级锁: 表级锁是锁定粒度最大的一种锁, 针对非索引字段加的锁。由于其锁定整个表的特性, 它的优点在于开销小、加锁快, 且不会出现死锁的情况。然而, 这也意味着并发度较低, 因为同时只有一个事务可以访问表。适用于当整个表都需要被操作时使用, 比如在表备份或表优化等场景中。**
3. **页级锁是介于表级锁和行级锁之间的一种锁。它通过锁定表中的某一页来实现, 可以在锁定多行数据的同时减少系统的开销。这种锁的开销和加锁时间都介于表级锁和行级锁之间, 并且也可能出现死锁的情况。页级锁适用于对数据块的操作, 尤其在某些存储引擎有页面大小限制的情况下更为常见。**

行级锁提供了最好的并发控制, 适合写密集型应用; 表级锁适用于读多写少的场景, 处理速度快但并发能力有限; 页级锁在这两者之间找到了一个平衡点。根据具体的业务需求和数据库操作的特点选择合适的锁定策略, 可以有效提升数据库的性能和并发处理能力。

MyISAM 仅仅支持表级锁(table-level locking), 一锁就锁整张表, 这在并发写的情况下性能非常差。InnoDB 不光支持表级锁(table-level locking), 还支持行级锁(row-level locking), 默认为行级

2.行级锁使用的注意事项

InnoDB 的行锁是针对索引字段加的锁, 表级锁是针对非索引字段加的锁。*当我们执行 `UPDATE`、`DELETE` 语句时, 如果 `WHERE` 条件中字段没有命中唯一索引或者索引失效的话, 就会InnoDB 会自动升级为表级锁, 以确保数据的一致性, 扫描全表对表中的所有行记录加表锁。******这个在我们日常工作开发中经常会遇到, 一定要多多注意!!!

3.行锁的类型

- **共享锁 (S 锁)**: 又称读锁, 允许多个事务对同一数据进行读取操作, 但禁止其他事务对该数据进行写操作。这种锁类型适用于那些仅需要读取数据并不打算修改数据的事务。
- **排他锁 (X 锁)**: 又称写锁/独占锁, 一旦某个事务对数据施加了排他锁, 其他任何事务都无法对该数据进行读或写, 直到该锁被释放

4.行锁的实现算法

行锁的实现主要有三类:

- Record Lock, 记录锁, 也就是仅仅把一条记录锁上;
 - 锁定单个数据库记录。当一个事务需要对某个数据行进行修改时, 它会请求一个记录锁。其他事务在尝试修改同一行时会被阻塞, 直到第一个事务完成。
- Gap Lock, 间隙锁, 锁定一个范围, 但是不包含记录本身;
 - 锁定两个索引记录之间的空隙。当一个事务需要在一个范围内插入新的记录时, 它会请求一个间隙锁。其他事务在这个范围内的插入操作会被阻塞, 直到第一个事务完成。

- Gap Locks锁定的记录之间或者记录与表的边界之间的“间隙”。例如，如果有记录(1, 'a')和(3, 'b')，那么(1, 'a')和(3, 'b')之间的空隙会被锁定，这意味着在(1, 'a')和(3, 'b')之间不能插入任何新的记录。
- Next-Key Lock: Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。其设计的目的是为了解决幻读
 - 是记录锁和间隙锁的结合。它锁定一个数据行及其前面的间隙。这种锁可以防止幻读（Phantom Read）的发生。
 - 当一个事务尝试插入、更新或删除一行时，InnoDB会自动获取该行的Next-Key Lock。这个锁不仅锁定该行，还锁定所有可能与该行有相同前缀的行，以防止幻读现象。具体来说，Next-Key Lock包含了：
 - 记录锁：锁定具体的行。
 - 间隙锁：锁定行之间的“空隙”，防止其他事务在这些空隙中插入新行。

4.共享锁、排他锁、意向锁

意向锁是表级别的锁，不会与行级的共享锁或排他锁互斥

按锁级别分类，有共享锁、排他锁和意向锁。

- **共享锁（S 锁）**：又称读锁，允许多个事务对同一数据进行读取操作，但禁止其他事务对该数据进行写操作。这种锁类型适用于那些仅需要读取数据并不打算修改数据的事务。
- **排他锁（X 锁）**：又称写锁/独占锁，一旦某个事务对数据施加了排他锁，其他任何事务都无法对该数据进行读或写，直到该锁被释放
- **意向锁**：意向锁的存在主要是为了解决多个事务并发操作时可能出现的加锁冲突问题。例如，如果一个事务已经锁定了表中的某一行，另一个事务想要锁定整张表，如果没有意向锁，第二个事务在锁定整张表前需要检查每一行是否已经被锁定，这样会导致效率低下。而有了意向锁之后，只需要检查意向锁即可快速判断是否可以进行表锁操作，从而大大提高了处理效率。意向共享锁表示一个事务有意设置共享锁或共享意向锁，而意向排他锁表示有意设置排他锁或排他意向锁。
 - **意向锁即为表级别的锁。设计目的主要是为了在一个事务中揭示下一行将被请求的锁类型。其支持两种意向锁：**
 - 意向共享锁（IS Lock），事务想要获得一张表中某几行的共享锁。
 - 意向排他锁（IX Lock），事务想要获得一张表中某几行的排他锁。

5.意向锁是什么有什么用

InnoDB存储引擎支持意向锁设计比较简练，其**意向锁即为表级别的锁。设计目的主要是为了在一个事务中揭示下一行将被请求的锁类型。其支持两种意向锁：**

意向共享锁（IS Lock），事务想要获得一张表中某几行的共享锁。

意向排他锁（IX Lock），事务想要获得一张表中某几行的排他锁

意向锁的存在主要是为了解决多个事务并发操作时可能出现的加锁冲突问题。例如，如果一个事务已经锁定了表中的某一行，另一个事务想要锁定整张表，如果没有意向锁，第二个事务在锁定整张表前需要检查每一行是否已经被锁定，这样会导致效率低下。而有了意向锁之后，只需要检查意向锁即可快速判断是否可以进行表锁操作，从而大大提高了处理效率。

意向锁是表级锁，共有两种：

- **意向共享锁（Intention Shared Lock, IS 锁）**：事务有意向对表中的某些记录加共享锁（S 锁），加共享锁前必须先取得该表的 IS 锁。
- **意向排他锁（Intention Exclusive Lock, IX 锁）**：事务有意向对表中的某些记录加排他锁（X 锁），加排他锁之前必须先取得该表的 IX 锁。

6.MySQL Update是行锁还是表锁？

MySQL的update操作既可以是行锁，也可以是表锁，具体使用哪种类型的锁，取决于执行update语句的条件，如果update语句中的where条件包含了索引列，并且只更新一条数据行，那这个时候就会加行锁，如果为了条件中不包含索引列，这个时候会加表锁。

7.读取已提交的情况下，使用select会不会加锁？

使用SELECT语句在读取已提交的情况下**可能会加锁**。

在数据库中，锁定机制是用于保证事务的隔离性和一致性的重要手段。当执行SELECT语句时，根据不同的事务隔离级别和具体的查询条件，数据库可能会对数据加上不同类型的锁。这些锁包括共享锁（S锁）和排他锁（X锁），以及间隙锁（gap lock）。具体如下：

1. **共享锁（S锁）**：当事务对数据进行读操作时，通常会加上共享锁，以防止其他事务对这些数据进行写操作。共享锁允许多个事务同时读取同一数据。
2. **排他锁（X锁）**：当事务需要对数据进行修改时，会加上排他锁，这防止其他事务对该数据进行读写操作。如果SELECT语句中使用了FOR UPDATE子句，或者在进行更新操作前，通常会加上排他锁。
3. **间隙锁（gap lock）**：在某些情况下，为了保护范围查询中的间隙，数据库可能会在索引记录之间加上间隙锁。这种锁可以防止其他事务在间隙中插入新的记录。

需要注意的是，在READ COMMITTED（读已提交）隔离级别下，事务读取的是记录版本链中的最新已提交版本。如果事务A的更新语句要修改的记录已经被其他事务加了X锁，事务A将不会对其加锁，因为它不满足更新语句中的搜索条件。

总的来说，使用SELECT语句是否加锁取决于事务的隔离级别、查询的具体条件以及是否使用了锁定选项。在实际应用中，了解这些锁定机制对于优化数据库性能和避免死锁是非常重要的。

7.mysql死锁

1.什么是mysql死锁？

MySQL的死锁是指**两个或更多的事务在执行过程中，因争夺资源而造成的一种相互等待的现象**。

具体来说，死锁产生在多个事务中，每个事务都持有至少一个资源，并且又都在等待其他事务释放它们所持有的资源。由于这些事务之间形成了循环等待的局面，没有任何一个事务能够继续前进，从而使得系统处于死锁状态。在数据库系统中，这种现象通常发生在并发访问相同数据时，特别是当多个事务试图对同一行数据进行修改时。

在MySQL中，死锁通常涉及到不同类型的锁，例如共享锁（读锁）和排他锁（写锁）。当事务A锁定了一行数据以进行修改，而事务B也试图修改这一行时，如果它们在锁定资源的先后顺序上存在差异，就可能发生死锁。此外，锁的升级操作也可能引发死锁，因为不同级别的锁可能有不同的兼容性和冲突规则。

2.(mysql)发生死锁的条件

MySQL发生死锁主要涉及以下四个条件：

1. **互斥**：资源是独占的且排他使用，进程互斥使用资源，即任意时刻一个资源只能给一个进程使用。
2. **请求与保持**：进程每次申请它所需要的一部分资源，在申请新的资源的同时，继续占用已分配到的资源。
3. **不可剥夺**：进程所获得的资源在未使用完毕之前，不被其他进程强行剥夺
4. **循环等待**：存在一个进程等待队列{P1,P2,..., Pn}，其中P1等待P2占有的资源，P2等待P3占有的资源，直至Pn等待P1占有的资源，形成一个进程等待环路。这样每一个进程都无法继续执行，因为它们都在等待下一个进程释放资源。

总的来说，以上四个条件通常一起出现才会导致死锁。在实际数据库操作中，合理设计事务逻辑和优化查询语句可以有效避免或减少死锁的发生。

3.解决死锁的方法：

解决死锁问题的方法包括**预防、避免、检测和解除死锁**。要有效应对死锁问题，需要从多个角度入手，采取不同的策略以确保系统稳定运行。以下是具体分析：

1. 预防死锁

- **资源一次性分配**：通过一次性分配所有资源给一个进程，可以破坏请求并保持条件，因为进程在启动执行前已经获得了所有所需资源，不会再有资源请求，这有助于防止死锁的发生[¹]。
- **资源有序分配法**：对资源进行编号，使进程按照编号顺序申请资源，这样能打破循环等待条件。这种方法虽然有效，但在实际运用中可能因为资源使用顺序限制而降低资源利用率[³]。
- **可剥夺资源**：允许系统在某些条件下剥夺进程的资源，这样可以破坏不可剥夺条件。当一个进程无法获得更多资源时，它释放已经持有的资源，供其他进程使用[¹]。

2. 避免死锁

- **银行家算法**：这是一种经典的避免死锁的算法，通过检查每次资源分配后系统是否处于安全状态来避免死锁。该算法通过维护两个数据结构，一个是总资源数，另一个是每个进程尚需的资源数，模拟资源分配，确保系统处于安全状态才真正分配资源[¹][²]。
- **资源预留**：系统对某些关键资源进行预留，不全部分配给进程，这样在关键时刻可以通过这些预留资源来避免死锁。这种方法适用于资源相对紧张的情况[⁴]。

3. 检测死锁

- **资源分配图简化法**：通过构建一个资源分配图，图中的节点和边分别表示进程和资源，通过化简这个图可以检测出是否存在死锁。如果能够完全简化这个图，说明没有死锁[¹]。
- **死锁检测算法**：定期运行死锁检测算法，一旦检测到死锁的存在，就采取相应的解除措施。这种方法允许死锁发生，但是能够及时发现并解决[⁵]。

4. 解除死锁

- **剥夺资源**：从死锁进程剥夺一些资源，将其分配给别的进程，以打破死锁状态。这种方法可能会造成部分工作丢失，影响进程的执行效率[¹]。
- **撤销进程**：直接撤销一些死锁进程或者选择撤销代价最小的进程，直到有足够的资源可以被重新分配，从而解除死锁状态。这是最后的手段，通常在其他方法无效时采用[¹][²]。
- **进程回退法**：通过检查点和回滚机制，将相关进程回退到之前的某个状态，重新开始执行，以此来解除死锁状态。这种方法需要系统具备检查点设置和恢复的功能[³]。

综上所述，解决死锁问题需要综合运用多种方法，根据系统的实际情况和需求灵活应用，才能确保系统高效、稳定地运行。

4.解决mysql死锁的方法

解决MySQL死锁问题涉及多个方面，包括**死锁检测、避免死锁的策略和具体处理方法**。下面将详细介绍解决MySQL死锁的方法：

• 超时

- 当两个事务互相等待时，当一个等待时间超过设置的某一阈值时，其中一个事务进行回滚，另一个等待的事务就能继续进行。

• 采用wait-for graph（等待图）的方式来进行死锁检测：

- **等待图是有向图，其节点代表事务，带有箭头的边则表示“等待”关系，箭头的方向指向被等待的事务。**

在每个事务请求锁并发生等待时都会判断是否存在回路，若存在则有死锁，通常来说InnoDB存储引擎选择回滚undo量最小的事务**

6.当前读和快照读有什么区别？

表记录有两种读取方式。

快照读

快照读(一致性非锁定读)：读取的是快照版本，读写不冲突。普通的 `SELECT` 就是快照读。通过mvcc来进行并发控制的，不用加锁。

只有在事务隔离级别 为读已提交和 可重复读下，InnoDB 才会使用快照读锁定读：

- 在 读已提交 级别下，对于快照数据，快照读总是读取被锁定行的最新一份快照数据，和当前读读取的数据是一样的，都是最新的。
- 在 可重复读级别下，对于快照数据，快照读总是读取本事务开始时的行数据版本，有可能读取的不是最新的数据。

mysql中的快照读是通过MVCC+undolog实现的。

当前读

当前读(一致性锁定读)：读取的是最新版本，但是读的时候不允许写，写的时候也不允许读。`UPDATE`、`DELETE`、`INSERT`` 是当前读。就是给行记录加 共享 锁或 排他锁。

Mysql实现当前读是通过共享锁+排他锁+Next-Key Lock(一种行锁)实现的。

每次对行数据进行读取的时候，加共享锁。此时就不允许修改，但是允许其他事务读取，所以每次都可以读到最新的数据。

每次对行数据进行修改的时候，加排他锁，不允许其他事务读取和修改。这种情况下其他事务读取的数据也一定是最新的数据。

每次对范围行数据进行读取的时候，对这个范围加一个范围共享锁。

每次对范围行数据进行修改的时候，读这个范围加一个范围排它锁。

基于上述锁机制，实现当前读，确保每次读取的都是最新的数据。

7.MVCC

1.什么是MVCC

MVCC (Multiversion concurrency control多版本并发控制)一种并发控制的方法，**用于在多个并发事务同时读写数据库时保持数据的一致性和隔离性。它是通过在每个数据行上维护多个版本的数据来实现的。当一个事务要对数据库中的数据进行修改时，MVCC 会为该事务创建一个数据快照，而不是直接修改实际的数据行。**

2.MVCC原理

InnoDB 的 MVCC 是通过 `read view` 和版本链实现的，版本链保存有历史版本记录，通过 `read view` 判断当前版本的数据是否可见，如果不可见，再从版本链中找到上一个版本，继续进行判断，直到找到一个可见的版本。

InnoDB默认的隔离级别是RR (REPEATABLE READ) ，RR解决脏读、不可重复读、幻读等问题，使用的是MVCC。MVCC全称Multi-Version Concurrency Control，即多版本的并发控制协议。它最大的优点是读不加锁，因此读写不冲突，并发性能好。InnoDB实现MVCC，多个版本的数据可以共存，主要基于以下技术及数据结构

隐藏列

InnoDB中每行数据都有隐藏列，隐藏列中包含了本行数据的事务id、指向undo log的指针等。

基于undo log的版本链

每行数据的隐藏列中包含了指向undo log的指针，而每条undo log也会指向更早版本的undo log，从而形成一条版本链

使用事务更新行记录的时候，就会生成版本链，执行过程如下：

1. 用排他锁锁住该行；
2. 将该行原本的值拷贝到 `undo log`，作为旧版本用于回滚；
3. 修改当前行的值，生成一个新版本，更新事务id，使回滚指针指向旧版本的记录，这样就形成一条版本链。

read view

通过隐藏列和版本链，MySQL可以将数据恢复到指定版本。但是具体要恢复到哪个版本，则需要根据ReadView来确定。所谓ReadView，是指事务（记做事务A）在某一时刻给整个事务系统（trx_sys）打快照，之后再行读操作时，会将读取到的数据中的事务id与trx_sys快照比较，从而判断数据对该ReadView是否可见，即对事务A是否可见

5.读写分离与主从同步

1.什么是读写分离？

读写分离主要是为了将对数据库的读写操作分散到不同的数据库节点上。这样的话，就能够小幅提升写性能，大幅提升读性能。

2.怎么读写分离？

不论是使用哪一种读写分离具体的实现方案，想要实现读写分离一般包含如下几步：

1. 部署多台数据库，选择其中的一台作为主数据库，其他的一台或者多台作为从数据库。
2. 数据库主机负责读写操作，从机只负责读操作。
3. 保证主数据库和从数据库之间的数据是实时同步的，这个过程也就是我们常说的**主从复制**。
4. 系统将写请求交给主数据库处理，读请求交给从数据库处理。

落实到项目本身的话，可以使用组件Sharding-JDBC。

Sharding-JDBC

Sharding-JDBC定位为轻量级java框架，在java的JDBC层提供的额外服务。它使用客户端直连数据库，以jar包形式提供服务，无需额外部署和依赖，可理解为增强版的JDBC驱动，完全兼容JDBC和各种ORM框架。

使用Sharding-JDBC可以在程序中轻松的实现数据库读写分离,优点在于数据源完全有Sharding托管，写操作自动执行master库，读操作自动执行slave库。不需要程序员在程序中关注这个实现了。

3.Mysql主从复制原理

主从同步:当master(主)库的数据发生变化的时候,变化会实时的同步到slave(从)库。

复制 (replication) 是MySQL数据库提供了一种高可用高性能的解决方案,一般用来建立大型的应用。总体来说, replication的工作原理分为以下3个步骤:

1. 主服务器 (master) 把数据更改记录到二进制日志 (binlog) 中。
2. 从服务器 (slave) 把主服务器的二进制日志复制到自己的中继日志 (relay log) 中。
3. 从服务器重做中继日志中的日志,把更改应用到自己的数据库上,以达到数据的最终一致性。

4.主从复制延迟问题

以 MySQL 为例,主从复制延迟可能达到 1 秒,如果有大量数据同步,延迟 1 分钟也是有可能的。主从复制延迟会带来一个问题:如果业务服务器将数据写入到数据库主服务器后立刻 (1 秒内) 进行读取,此时读操作访问的是从机,主机还没有将数据复制过来,到从机读取数据是读不到最新数据的,业务上就可能出现数据不一致的问题。例如,用户刚注册完后立刻登录,业务服务器会提示他“你还没有注册”,而用户明明刚才已经注册成功了。

解决主从复制延迟有几种常见的方法:

- **读从机失败后再读一次主机**

这就是通常所说的“二次读取”,二次读取和业务无绑定,只需要对底层数据库访问的 API 进行封装即可,实现代价较小,不足之处在于如果有很多二次读取,将大大增加主机的读操作压力。例如,黑客暴力破解账号,会导致大量的二次读取操作,主机可能顶不住读操作的压力从而崩溃。

- **关键业务读写操作全部指向主机,非关键业务采用读写分离**

例如,对于一个用户管理系统来说,注册 + 登录的业务读写操作全部访问主机,用户的介绍、爱好、等级等业务,可以采用读写分离,因为即使用户改了自己的自我介绍,在查询时却看到了自我介绍还是旧的,业务影响与不能登录相比就小很多,还可以忍受。

6.分库分表

1.为什么要分库分表?

读写分离分散了数据库读写操作的压力,但没有分散存储压力,当数据量达到千万甚至上亿条的时候,单台数据库服务器的存储能力会成为系统的瓶颈,主要体现在这几个方面:

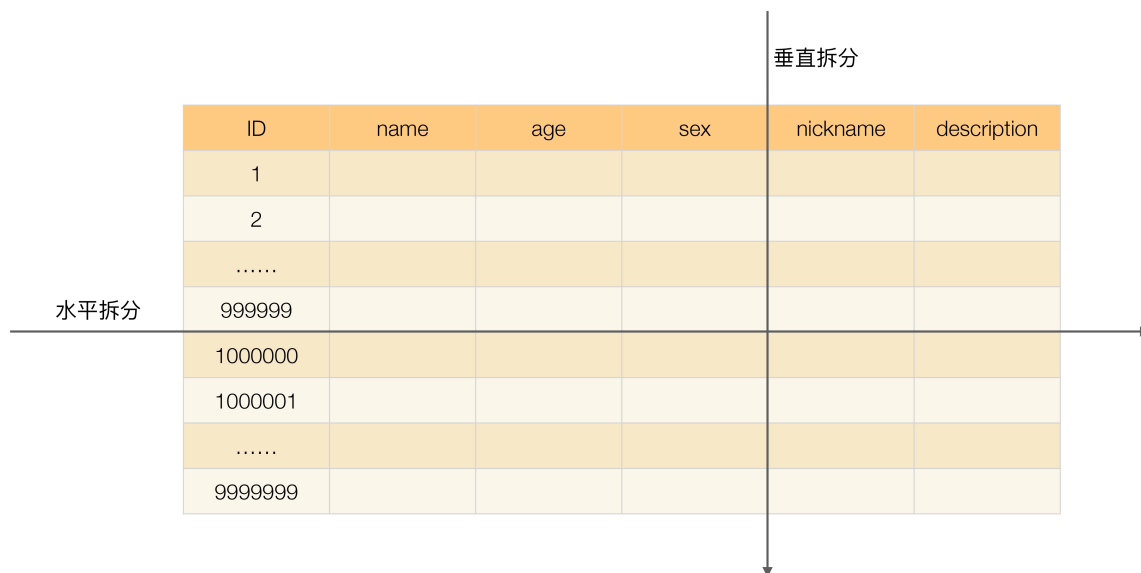
- 数据量太大,读写的性能会下降,即使有索引,索引也会变得很大,性能同样会下降。
- 数据文件会变得很大,数据库备份和恢复需要耗费很长时间。数据文件越大,极端情况下丢失数据的风险越高 (例如,机房火灾导致数据库主备机都发生故障)。

基于上述原因,单个数据库服务器存储的数据量不能太大,需要控制在一定的范围内。为了满足业务数据存储的需求,就需要将存储分散到多台数据库服务器上。今天我来介绍常见的分散存储的方法“分库分表”,其中包括“分库”和“分表”两大类。

2.分库分表的4种策略

业务分库:按照业务模块将数据分散到不同的数据库服务器

水平分表和垂直分表:



4.分库分表的使用方案

ShardingSphere 绝对可以说是当前分库分表的首选！ShardingSphere 的功能完善，除了支持读写分离和分库分表，还提供分布式事务、数据库治理、影子库、数据加密和脱敏等功能。

5.分库分表后，数据怎么迁移呢？

分库分表之后，我们如何将老库（单库单表）的数据迁移到新库（分库分表后的数据库系统）呢？

比较简单同时也是非常常用的方案就是**停机迁移**，写个脚本老库的数据写到新库中。比如你在凌晨 2 点，系统使用的人数非常少的时候，挂一个公告说系统要维护升级预计 1 小时。然后，你写一个脚本将老库的数据都同步到新库中。

如果你不想停机迁移数据的话，也可以考虑**双写方案**。双写方案是针对那种不能停机迁移的场景，实现起来要稍微麻烦一些。具体原理是这样的：

- 我们对老库的更新操作（增删改），同时也要写入新库（双写）。如果操作的数据不存在于新库的话，需要插入到新库中。这样就能保证，咱们新库里的数据是最新的。
- 在迁移过程，双写只会让被更新操作过的老库中的数据同步到新库，我们还需要自己写脚本将老库中的数据和新库的数据做比对。如果新库中没有，那咱们就把数据插入到新库。如果新库有，旧库没有，就把新库对应的数据删除（冗余数据清理）。
- 重复上一步的操作，直到老库和新库的数据一致为止。

想要在项目中实施双写还是比较麻烦的，很容易会出现问题。我们可以借助上面提到的数据库同步工具 Canal 做增量数据迁移（还是依赖 binlog，开发和维护成本较低）。